MirrorBench: A Benchmark to Evaluate Conversational User-Proxy Agents for Human-Likeness

Ashutosh Hathidara*
SAP Labs
Palo Alto, California, USA
ashutosh.hathidara@sap.com

Julien Yu
SAP Labs
Palo Alto, California, USA

Vaishali Senthil
SAP Labs
Palo Alto, California, USA

Sebastian Schreiber
SAP Labs
Palo Alto, California, USA

Anil Babu Ankisettipalli
SAP Labs
Palo Alto, California, USA

Abstract

Large language models (LLMs) are increasingly used as human simulators, both for evaluating conversational systems and for generating fine-tuning data. However, naive "act-as-a-user" prompting often yields verbose, unrealistic utterances, motivating principled evaluation of *user proxy agents*. We present MirrorBench, a reproducible and extensible benchmarking framework that evaluates user proxies solely on their ability to produce human-like user utterances across diverse conversational regimes, explicitly decoupled from downstream task success. MirrorBench combines three lexical-diversity metrics (MATTR, Yule's K, and HD-D) with three LLM-judge-based metrics (GTEval, Pairwise Indistinguishability, and Rubric-and-Reason), and contextualizes judge scores using Human–Human and Proxy–Proxy calibration controls. Across four public datasets, MirrorBench yields variance-aware comparisons and reveals systematic gaps between user proxies and real human users. The framework is open source[1] and includes a command-line interface for running and managing user-proxy benchmarking experiments.

CCS Concepts

• **Computing methodologies** → **Intelligent agents**; *Discourse, dialogue and pragmatics*; **Lexical semantics**; *Natural language generation*; *Simulation evaluation.*

Keywords

Large Language Models (LLMs), User-proxy agents, Human-likeness benchmarking, LLM-as-a-judge, Judge sensitivity, Human–LLM correlation, Lexical diversity, MATTR, HD-D, Yule's K, Pairwise indistinguishability, GTEval, Rubric-And-Reason, ChatbotArena, ClariQ, QULAC, OASST1

[1] https://github.com/SAP/mirrorbench

KDD, Jeju, Korea

ACM ISBN 978-1-4503-XXXX-X/2026/08
https://doi.org/XXXXXXX.XXXXXXX

ACM Reference Format:
Ashutosh Hathidara, Julien Yu, Vaishali Senthil, Sebastian Schreiber, and Anil Babu Ankisettipalli. 2026. MirrorBench: A Benchmark to Evaluate Conversational User-Proxy Agents for Human-Likeness. In *Proceedings of ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD).* ACM, New York, NY, USA, 21 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Deploying and improving conversational AI hinges on evaluation under realistic user interactions. Expert human-annotated utterances, while authoritative, is costly and difficult to scale for systematic evaluation. At the same time, the instruction-tuning of models for production systems demands large volumes of authentic user–assistant exchanges. To meet both needs, practitioners increasingly rely on *user proxy agents*, namely LLMs prompted to emulate specified human personas, to synthesize user utterances for automated testing and post-training at scale.

In practice, user proxies are now used far beyond toy role-play: they drive regression testing for assistant updates [37], evaluate tool-use and grounding behaviors [39], generate synthetic dialogues for post-training [12, 28], and enable stress tests that are infeasible with human participants [25, 40]. However, naive "act-as-a-user" prompting often yields verbose, overly cooperative utterances that diverge from real user behavior [42], indicating that unrealistic proxies can bias downstream evaluations and introduce distribution shift when used for data generation. This motivates a central question: *How can we rigorously measure the human-likeness of a user proxy, independently of downstream task success?* Here, we define a *user* as a human interlocutor participating in a specific conversational or task-oriented dialogue.

We introduce MirrorBench, a reproducible and extensible framework for *variance-aware* user-proxy benchmarking on human-likeness metrics using multiple human reference datasets across diverse conversational settings. MirrorBench evaluates proxies solely on human-likeness and deliberately separates this from task completion. We define the *human-likeness* of a user proxy as the extent to which its generated user utterances match real human user utterances along two complementary axes: (i) *lexical similarity*, operationalized via human-anchored lexical-diversity statistics (e.g., MATTR, HD-D, Yule's K), and (ii) *behavioral realism*, operationalized via LLM-judge evaluations that assess whether the proxy's user turns *sound* like a real user in context (e.g., naturalness, tone, appropriateness).

Human-likeness is inherently scenario-dependent: what sounds realistic in open-domain chat may be unrealistic in information-seeking or clarification-heavy interactions. Accordingly, MirrorBench does not aim to capture an unconstrained notion of "general" human behavior. Instead, we evaluate *utterance-level realism conditioned on the same task context*, asking whether a proxy produces user turns that resemble those of real users *in that regime*. This framing explicitly decouples user-proxy quality from assistant task success, and treats human-likeness as a measurable property of the *user side* of the interaction under controlled conditions.

To evaluate the realism of user–proxy agents against real-world dialogues, MirrorBench packages four open-source corpora preprocessed for user-proxy evaluation: QULAC [3], ClariQ [2], OASST1 [18], and ChatbotArena [41]. To quantify alignment in lexical diversity with human users, we implement Moving-Average Type-Token Ratio (MATTR) [9], Yule's K [34], and a Hypergeometric Distribution Diversity (HD-D) [7]. To assess behavioral realism, we employ a family of LLM-judge metrics, including GTEval [43], Pairwise Indistinguishability (PI) [41], and Rubric-and-Reason (RNR) [22]. Taken together, these metrics quantify the realism of the *user* side of the dialogue, explicitly decoupled from assistant capabilities and task success.

Our benchmark reveals that even strong LLMs exhibit systematic gaps from real users, and that these gaps vary substantially across conversational regimes. Across datasets, we observe a recurring realism-diversity tension: proxies that score highly on judge-based behavioral realism can still under-match human lexical/distributional characteristics in clarification-centric settings, motivating the use of complementary metric families. We also find that absolute judge scores and, in some cases, fine-grained orderings can shift with the judge choice, reinforcing the importance of transparent multi-judge reporting and calibration controls when comparing closely matched proxies.

In summary, our contributions are as follows: (1) a benchmark protocol for evaluating user-proxy agents for human-likeness with human-anchored normalization and calibrated judge metrics; (2) six metrics spanning lexical-diversity (MATTR, HD-D, Yule's K) and judge-based realism (GTEval, Pairwise Indistinguishability, Rubric-and-Reason); (3) a unified preprocessing of four open conversational datasets for user-proxy benchmarking; and (4) an empirical study across six user-proxy LLMs, yielding actionable insights on realism-diversity tensions and judge sensitivity.

2 Related Work

User simulation and user proxies. Simulating users to train and evaluate dialogue agents has a long history in NLP [5]. Early user simulators were typically goal-driven or rule-based, using predefined agendas over dialogue acts to approximate human behavior [33]. More recently, *LLM-based* user proxies have enabled open-ended interactions that better match real conversational variability, but they also introduce failure modes such as verbosity, role drift, and inconsistent intent adherence. For instance, [36] proposes a User Simulator with Implicit Profiles (USP) that infers latent user traits to improve realism beyond naive role-play prompting. LLM user proxies are increasingly used as evaluation drivers in interactive testbeds (e.g., tool-use and multi-turn assistant evaluation) [1, 12, 24], and related systems generate synthetic human–assistant conversations from seed dialogues [43]. In parallel, broader agent evaluation frameworks (e.g., AgentBench [21] and AgentScope [31]) evaluate *agents in environments* but do not isolate the human-likeness of the user role.

User-proxy evaluation. SimulatorArena [10] investigates whether profile-conditioned LLM simulators can *replace human judges* for ranking AI assistants, demonstrating strong rank correlation on two narrow task types (math tutoring, document creation). SimUSER [6] builds persona-based user agents for recommender system evaluation, measuring non-conversational behavioral alignment (clicks, engagement, etc.) at micro and macro levels. Both works treat the simulator as a downstream evaluation tool rather than as a subject of study in its own right. Critically, neither measures *lexical diversity* of generated user turns nor examines the tension between realism and diversity, a key finding in our work, where judge-based indistinguishability and human-anchored vocabulary alignment can diverge substantially across datasets and models.

LLMs as judges. LLM-as-a-judge evaluation prompts a strong model to rate or compare dialogue quality, reducing reliance on reference answers or large-scale human annotation [20, 23]. Systems such as MT-Bench/ChatbotArena [41], G-Eval [23], and rubric-driven evaluators such as Prometheus [16] have enabled scalable, flexible assessments by operationalizing custom criteria in natural language prompts. However, most LLM-judge setups focus on evaluating *assistant* outputs, whereas our setting repurposes judge-based evaluation to assess the *user* side and pairs it with explicit calibration controls (HH/PP) and complementary lexical/distributional metrics to better contextualize score magnitudes across datasets and model families.

3 MirrorBench

This section describes the MirrorBench evaluation methodology for evaluating user-proxy LLMs across diverse conversational datasets. Figure 1 summarizes the MirrorBench pipeline. We start from diverse conversational datasets and synthesize user goals to drive controlled, goal-conditioned rollouts between a user-proxy LLM and an assistant LLM under explicit role separation. The resulting transcripts are scored with a combined evaluation suite of judge-based behavioral realism metrics and lexical/distributional metrics, with Human–Human (HH) and Proxy–Proxy (PP) controls used to contextualize judge scores. Finally, we run robustness analyses, swapping assistants or judges and repeating runs across seeds, to quantify sensitivity and variance before reporting benchmark outputs such as leaderboards, reliability checks, and operational profiles. Detailed system architecture is described in Appendix C.

3.1 Benchmark Protocol

We formalize evaluation of a *user–proxy agent* θ_u on a domain-agnostic dialogue dataset D^{ref} with respect to a set of human-likeness metrics M. Let

$$D^{\text{ref}} = \{ d_j^{\text{ref}} \in \mathcal{D} \}_{j=1}^{n}$$

denote a corpus of reference, human-grounded dialogues (where $\mathcal{D}$ is the set of finite dialogues). Each dialogue is a sequence of

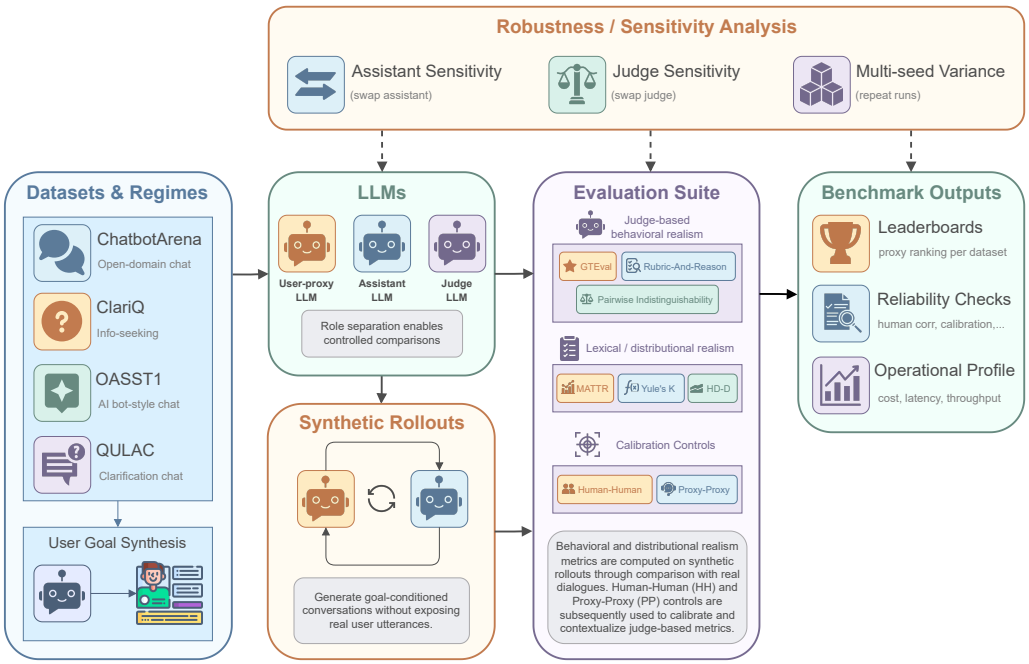

Figure 1: MirrorBench benchmark pipeline, spanning dataset preparation, synthetic rollouts, the evaluation suite, and an overlay of robustness/sensitivity analysis.

user/assistant turns,

$$d_j^{\text{ref}} = \left[(u_{j,1}^{\text{ref}}, a_{j,1}^{\text{ref}}), \ldots, (u_{j,L_j}^{\text{ref}}, a_{j,L_j}^{\text{ref}})\right],$$

with variable length $L_j \geq 1$. Here $u_{j,t}^{\text{ref}}$ is a *human* user utterance at turn t, and $a_{j,t}^{\text{ref}}$ is the corresponding assistant reply (curated by a human expert or generated by an LLM in the source dataset).

Goal specification. In some datasets, each reference dialogue d_j^{ref} includes an explicit user-goal annotation $g_j \in \mathcal{G}$. When no goal is provided, we derive one using an auxiliary goal generator $\theta_{\text{goal}} : \mathcal{D} \to \mathcal{G}$ applied to d_j^{ref}:

$$g_j = \begin{cases} \text{given goal attached to } d_j^{\text{ref}}, & \text{if available,} \\ \theta_{\text{goal}}(d_j^{\text{ref}}), & \text{otherwise.} \end{cases}$$

The goal specification is used to condition θ_u when synthesizing user utterances; if D^{ref} already provides such annotations, the goal-inference step is omitted.

Synthetic rollouts. To compare a user–proxy agent's behavior against human references, we synthesize proxy–assistant dialogues by rolling out the user–proxy θ_u against an assistant model θ_a. For each reference dialogue d_j^{ref} with user goal g_j, we condition θ_u on g_j and the accumulated dialogue history to produce user turns $\hat{u}_{j,t}$, while θ_a produces contextual assistant replies $\hat{a}_{j,t}$. Here, we condition θ_a on d_j^{ref} such that the synthetic rollout trajectory follows the same path as real one. This is a deliberate design choice: anchoring the assistant to the reference history keeps the synthetic rollout grounded on the same conversational trajectory as the reference dialogue, ensuring that the proxy and the real user are compared under identical context. This yields a synthetic transcript

$$\hat{d}_j = \left[(\hat{u}_{j,1}, \hat{a}_{j,1}), \ldots, (\hat{u}_{j,L_j}, \hat{a}_{j,L_j})\right],$$

which we compare with the corresponding reference dialogue d_j^{ref} to score human-likeness of the user-proxy. Importantly, our evaluation targets the user side $\{\hat{u}_{j,t}\}_{t=1}^{L_j}$ only: we consider the behavior, tone, and style of the proxy's utterances, and do not score assistant response content.

Task formalization. Let T denote the fully specified, realism-evaluation task for the *human-likeness* of a user-proxy agent θ_u over the reference corpus D^{ref}. The task encapsulates all configuration required for execution and metric aggregation, ensuring that the evaluation process is reproducible and self-contained.

Inputs. The inputs to T include a reference dataset D^{ref}, an optional goal generator θ_{goal} (used when no explicit goal is available), the user-proxy agent under test θ_u, an assistant model θ_a used for synthetic rollouts, and a set of human-likeness metrics M.

Outputs. The evaluation can be expressed as a composite executable function

$$R_{\theta_u} = \Psi_T(\theta_u, D^{\text{ref}}, M; \theta_a, \theta_{\text{goal}}),$$

where Ψ_T denotes the instantiated evaluation pipeline. This pipeline performs rollout generation by simulating synthetic dialogues $\hat{D} = \{\hat{d}_j\}_{j=1}^{n}$ between θ_u and θ_a under the same dialogue structures and goals as D^{ref}. It then applies the metric suite M to each dialogue pair $(\hat{d}_j, d_j^{\text{ref}})$, and aggregates the resulting scores across dialogues.

Given a user-proxy agent θ_u, the output R_{θ_u} is a structured evaluation record containing per-sample metric values, aggregated results with confidence intervals, and auxiliary metadata such as random seeds, model identifiers, and telemetry reports. We also report metric-level calibration and reliability diagnostics that quantify metric stability and judge variability.

3.2 Datasets & Tasks

We evaluate user proxies across four open-source conversational datasets spanning diverse domains and interaction patterns, collectively providing **795** conversations with real human user turns, enabling direct comparison between proxy-generated and real user behavior.

Table 1: Datasets used for the evaluation. All datasets are open-source and include real human user utterances.

Dataset	Samples	Avg Turns
ChatbotArena [41]	195	2.5
ClariQ [2]	200	7.0
OASST1 [18]	200	3.3
QULAC [3]	200	2.0

In Table 1, notice that the number of samples we use for benchmarking does not exhaust all the samples of the underlying open-source datasets. Instead, we perform *stratified* sampling over the original datasets to extract the final evaluation datasets.

For each dataset, we first normalize conversations into alternating user-assistant turn sequences, retaining only English dialogues with at least two turns. We then define dataset-specific stratification keys to ensure broad coverage: OASST1 & ChatbotArena conversations are bucketed by the number of user turns (short, medium, long), QULAC entries are grouped by topic type & facet category, and ClariQ dialogues are distributed across topic buckets & clarification-pair counts. Within each stratum, we allocate samples proportionally to the population size while enforcing a minimum-per-stratum threshold to prevent underrepresentation. Using a fixed random seed for reproducibility, we sample up to 200 samples per dataset, yielding a balanced benchmark that spans diverse conversational structures, topic domains, and interaction patterns without oversampling any single mode.

For each conversation in all four datasets, we generate a user goal description using an auxiliary LLM (θ_g defined in § 3.1) that summarizes the user's intent, behavior, tone, and persona based on the real conversation. This description serves as the initialization prompt for user proxies during evaluation.

3.3 Metrics

MirrorBench quantifies the human-likeness of user proxies with two metric families: (i) *lexical-diversity metrics* measure vocabulary richness and repetition patterns (§ 3.3.1), and (ii) *judge-based realism metrics* measure behavioral realism (Section 3.3.2). All metrics are *human-anchored*: proxy scores are compared against the empirical distribution of real-user utterances from the *same* dataset and under same tokenization, yielding interpretable, calibrated results (implementation details in Appendix C.3).

3.3.1 Lexical Diversity Metrics. Lexical-diversity metrics capture how varied a user proxy's word choices are and how often they repeat tokens. Because raw values are sensitive to sequence length, domain, and tokenization, we normalize (z-score) proxy scores with respect to the human distribution on the same dataset.

Moving Average Type-Token Ratio (MATTR). The classical type–token ratio (TTR) [8, 13] decreases as sequence length grows. The moving-average TTR (MATTR) mitigates this length bias by averaging TTR over a sliding window of width w [9]. Let types$(\cdot)$ return the set of distinct token types in its argument. For a token sequence $\mathbf{t} = (t_1, \ldots, t_N)$ and window size $w \le N$,

$$\mathrm{MATTR}_w(\mathbf{t}) \;=\; \sum_{i=1}^{N-w+1} \frac{\left|\mathrm{types}(t_i, \ldots, t_{i+w-1})\right|}{w(N-w+1)}, \tag{1}$$

where types$(\cdot)$ returns the set of distinct token types in the window. We use $w = 50$ by default unless stated otherwise. Note that for *Qulac* dataset, human reference sequences are very short, so MATTR degrades to plain TTR; lexical diversity scores for this dataset should be interpreted accordingly.

Yule's K. Yule's characteristic constant K [34] summarizes repetitiveness from the token-frequency spectrum. Let V_i be the number of types occurring exactly i times, and $N = \sum_{i\ge1} iV_i$ the token count. Then

$$K(\mathbf{t}) \;=\; 10^4\,\frac{\sum_{i\ge1} i^2 V_i \;-\; N}{N^2}. \tag{2}$$

Lower K indicates richer, less repetitive text; higher K indicates greater repetition. The factor 10^4 provides a convenient numeric range.

Hypergeometric Distribution Diversity (HD-D). HD-D [7] estimates vocabulary diversity via hypergeometric sampling, offering robustness to length variation. It approximates the expected number of *distinct* types observed in a sample of size s drawn *without replacement* from a token sequence $\mathbf{t} = (t_1, \ldots, t_N)$. For a type i with frequency f_i in $\mathbf{t}$, the probability that the sample contains at least one instance of that type is

$$P(\text{type } i \text{ in sample}) \;=\; 1 \;-\; \frac{\binom{N-f_i}{s}}{\binom{N}{s}}, \quad 1 \le s \le N,$$

and the HD-D score is their normalized sum:

$$\text{HD-D}_s(\mathbf{t}) \;=\; \frac{1}{s}\sum_{i=1}^{V}\left(1 \;-\; \frac{\binom{N-f_i}{s}}{\binom{N}{s}}\right), \tag{3}$$

where $V = \left|\mathrm{types}(\mathbf{t})\right|$. Following the VOCD-D convention, we use $s = 42$ by default [26].

Human-Anchored Z-Score Normalization. Raw lexical-diversity scores are dataset-dependent; we therefore report human-anchored z-scores computed on the *same* split and tokenizer. Let $\mathcal{H} = \{\mathbf{t}_1^{\mathrm{ref}}, \ldots, \mathbf{t}_n^{\mathrm{ref}}\}$ be human user-side token sequences, where each $\mathbf{t}_i^{\mathrm{ref}}$ concatenates the human user turns of one reference dialogue. For any $m \in \{\mathrm{MATTR}_w, K, \text{HD-D}_s\}$, let $\mu_m^{\mathrm{hum}} = \frac{1}{|\mathcal{H}|}\sum_{\mathbf{t}\in\mathcal{H}} m(\mathbf{t})$ and $\sigma_m^{\mathrm{hum}} = \sqrt{\frac{1}{|\mathcal{H}|-1}\sum_{\mathbf{t}\in\mathcal{H}}\big(m(\mathbf{t}) - \mu_m^{\mathrm{hum}}\big)^2}$. During evaluation, each rollout episode

produces a synthetic proxy-assistant dialogue, which we denote by $\hat{d}_i$ (§ 3.1). From $\hat{d}_i$, we extract only the *proxy user* turns (i.e., the outputs of the user–proxy agent θ_u) and concatenate them in temporal order to obtain a proxy user token sequence $\hat{\mathbf{t}}_i$. This $\hat{\mathbf{t}}_i$ is the proxy-side analogue of the human sequence $\mathbf{t}_i^{\text{ref}}$ in $\mathcal{H}$. We then assign that rollout episode a human-anchored standardized lexical score:

$$z_m(\hat{\mathbf{t}}_i) \;=\; \frac{m(\hat{\mathbf{t}}_i) - \mu_m^{\text{hum}}}{\sigma_m^{\text{hum}}}.$$

By design, $z_m \approx 0$ indicates that the proxy's lexical behavior matches the human mean for metric m. The sign of z_m is metric-dependent: for MATTR_w and HD-D_s, $z_m > 0$ corresponds to *greater* lexical diversity than the human average; for Yule's K, $z_m > 0$ corresponds to *more* repetition (i.e., *lower* diversity). For reporting at the dataset level, we aggregate across all rollout dialogues $\{\, \hat{d}_i \in \hat{D} \,\}_{i=1}^{n}$ executed under a fixed configuration. Let $\{z_m(\hat{\mathbf{t}}_1), \ldots, z_m(\hat{\mathbf{t}}_n)\}$ be the per-episode standardized scores for metric m. We report their sample mean $\bar{z}_m$ together with a two-sided 95% confidence interval derived from the Student-t distribution:

$$\text{CI}_{0.95}(m) \;=\; \bar{z}_m \;\pm\; t_{0.975,\,n-1}\,\frac{s_m}{\sqrt{n}}, \tag{4}$$

where s_m is the unbiased sample standard deviation, and $t_{0.975,\,n-1}$ is the 97.5th percentile of the t-distribution with $n-1$ degrees of freedom. These aggregated statistics, namely $\bar{z}_m$, s_m, and $\text{CI}_{0.95}(m)$, are the quantities that MirrorBench reports for downstream comparison across user proxies, datasets, and seeds.

3.3.2 Judge-Based Realism Metrics. Lexical diversity alone cannot capture whether a simulated user *feels human-like*. Human-likeness also depends on discourse phenomena such as tone, politeness, hesitations, or style. MirrorBench therefore includes *judge-based realism metrics*: LLM evaluators that score proxy behavior against human references along higher-level dimensions. Each judge call uses a chain-of-thought style prompting strategy [38]: the judge is instructed to reason privately and then produce a final structured verdict. To improve robustness, each judge is executed with a self-consistency parameter $c \geq 1$, which repeats the judgment c times under the same prompt and aggregates the resulting scores.

GTEval (Relative Realism Scoring). GTEval [43] compares a proxy-generated user transcript against a human reference and returns a similarity score in $[0, 1]$, where higher values indicate that the proxy is judged to behave more like the human reference. The judge model (with $c = 1$) is shown the pair $(\hat{d}_j, d_j^{\text{ref}})$ and asked to assess stylistic and behavioral similarity (e.g., tone, naturalness). Let $\text{J}^{(r)}(\cdot) \in [0, 1]$ denote the scalar verdict returned by the judge on repetition r. The GTEval score for episode j is

$$s_j^{(\text{GTEval})} \;=\; \frac{1}{c}\sum_{r=1}^{c} \text{J}^{(r)}(\hat{d}_j,\, d_j^{\text{ref}}), \tag{5}$$

where each $\text{J}^{(r)}$ corresponds to an independent call to the judge with a different random seed.

Pairwise Indistinguishability (PI). Here, judge LLM is given two anonymized user-assistant conversations, and asked to choose one in which the user sounds more like a human. For each sample, we present one proxy conversation $\hat{d}_j$ and one human reference d_j^{ref} in random order, unlabeled except as A or B. The judge returns a verdict $v_{j,r} \in \{\text{A}, \text{B}, \text{Tie}\}$ for judgment round r. Let $p_{j,r} \in \{\text{A}, \text{B}\}$ be the position where the proxy was shown for that comparison (randomized per judgment). With c repeated judgments (default $c = 3$), we define the per-episode win rate

$$w_j^{(\text{PI})} \;=\; \frac{1}{c}\sum_{r=1}^{c}\left(\mathbb{1}[v_{j,r} = p_{j,r}] \;+\; 0.5\,\mathbb{1}[v_{j,r} = \text{Tie}]\right). \tag{6}$$

Aggregated win rate is summarized by the mean $\bar{w}^{(\text{PI})} = \frac{1}{n}\sum_{j=1}^{n} w_j^{(\text{PI})}$ and its centered form $\Delta w^{(\text{PI})} = \bar{w}^{(\text{PI})} - 0.5$. Here, $\Delta w > 0$ indicates the judge tends to prefer the proxy user over the human baseline, $\Delta w < 0$ favors real users, and $\Delta w \approx 0$ indicates that the proxy is effectively *indistinguishable* from the human baseline.

Rubric-and-Reason (RNR). RNR adapts rubric-style LLM evaluation to judge human-likeness without requiring a paired human reference. The judge sees only the proxy-side user transcript $\hat{d}_j$ and a rubric that defines realism along dimensions such as style, behavior, and tone. For a given rubric, the judge returns a binary verdict $v^{(r)} \in \{\text{YES}, \text{NO}\}$ on repetition r, which we map to

$$\hat{s}_{\text{RNR}}^{(r)}(\hat{d}_j) \;=\; \begin{cases} 1.0, & \text{if } v^{(r)} = \text{YES}, \\ 0.0, & \text{if } v^{(r)} = \text{NO}. \end{cases}$$

We repeat each judgment $c = 2$ times. The aggregate RNR score for sample j is

$$s_{\text{RNR}}(\hat{d}_j) \;=\; \frac{1}{c}\sum_{r=1}^{c} \hat{s}_{\text{RNR}}^{(r)}(\hat{d}_j). \tag{7}$$

Unlike GTEval and PI, which are explicitly comparative, RNR is reference-free and produces an absolute realism score for the proxy alone. Because this absolute score depends on the judge model and the elaborated rubric, it is interpreted *relatively*: higher $s_{\text{RNR}}(\hat{d}_j)$ means the proxy is behaving more like a real user under the fixed judge and rubric set. Optionally, we also evaluate the human reference d_j^{ref} under the same procedure to obtain $s_{\text{RNR}}(d_j^{\text{ref}})$, which serves as an empirical upper bound (useful for calibration).

Calibration via HH and PP Controls. Judge models can exhibit systematic bias (e.g., favoring verbosity, excessive politeness, or self-preference). To expose this bias and to place GTEval and PI scores on an interpretable scale, we optionally compute two control conditions:

- **Human-Human (HH):** Compare a human conversation to itself, $s_{\text{HH},j} = \text{Judge}(d_j^{\text{ref}}, d_j^{\text{ref}})$. This approximates the judge's effective ceiling for genuine human behavior on episode j.
- **Proxy-Proxy (PP):** Compare a proxy conversation to itself, $s_{\text{PP},j} = \text{Judge}(\hat{d}_j, \hat{d}_j)$. This estimates how the judge scores the proxy in isolation.

For GTEval, both $s_{\text{HH},j}$ and $s_{\text{PP},j}$ are typically expected to be close to 1. For PI, a self comparison should yield an ideal effective win rate near 0.5. We calculate their empirical means across all n samples: $\mu_{\text{HH}} = \frac{1}{n}\sum_{j=1}^{n} s_{\text{HH},j}$, $\mu_{\text{PP}} = \frac{1}{n}\sum_{j=1}^{n} s_{\text{PP},j}$. For PI Metric, we also calculate calibrated score s_{cal}. Let s_{raw} denote the uncalibrated proxy score of interest (e.g. $\bar{w}^{(\text{PI})}$). We define a calibrated $[0, 1]$ score via

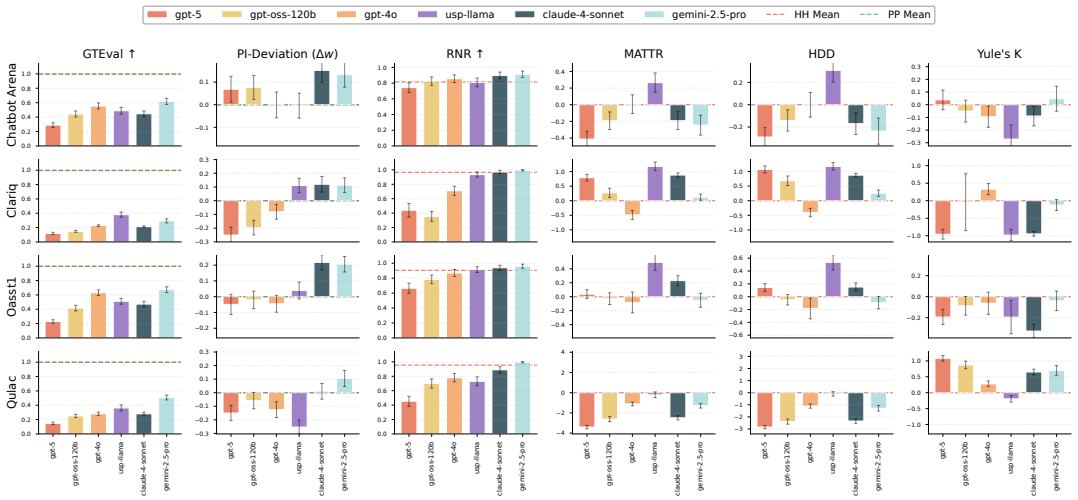

Figure 2: Human-likeness of six user-proxy LLMs across four datasets. Higher is better for judge-based metrics (GTEval, PI, RNR). Lexical-diversity metrics are z-scored to human baselines (0 is best). We fix the judge to Claude-4-Sonnet and the assistant to GPT-4o.

an affine rescaling between the PP and HH anchors:

$$s_{\text{cal}}^{PI} = \text{clip}\left(\frac{s_{\text{raw}} - \mu_{\text{PP}}}{\max(\epsilon,\ \mu_{\text{HH}} - \mu_{\text{PP}})},\ 0,\ 1\right), \tag{8}$$

where $\epsilon = 10^{-6}$ prevents division by zero and $\text{clip}(\cdot, 0, 1)$ bounds the result. Under this normalization, $s_{\text{cal}} \approx 0$ means the proxy is no better than its own PP baseline, while $s_{\text{cal}} \approx 1$ means the proxy is judged as indistinguishable from human under the same judge.

4 Experiments & Results

We evaluate multiple user-proxy LLMs across four diverse conversational datasets, ChatbotArena, ClariQ, OASST1, and QULAC, to quantify human-likeness using both lexical-diversity and LLM-judge realism metrics. Concretely, we report MATTR, Yule's K, and HD-D (human-anchored z-scores), together with three judge-metrics GTEval, PI, and RNR. Our experiments (i) measure how closely different proxy architectures reproduce human user behavior, (ii) assess the reliability and calibration of LLM-as-judge metrics, and (iii) characterize computational cost, latency, and throughput under large-scale runs.

We compare six LLMs as user proxies: GPT-4o [14], GPT-5 [29], GPT-OSS-120B [30], Claude-4-Sonnet [4], Gemini-2.5-Pro [11], and a specialized open-source user-simulation proxy (USP) based on LLaMA [36]. Unless otherwise noted, the assistant is fixed to GPT-4o and the judge to Claude-4-Sonnet for all primary results, enabling apples-to-apples comparisons across proxies. All the evaluations are performed with a single seed (unless otherwise specified), and we report aggregated scores with 95% confidence intervals. Judge scores are calibrated, and we include a human correlation check to validate judge trends. We also analyze fine-grained telemetry to make cost/performance trade-offs explicit.

4.1 Comparison of User-Proxy LLMs

Figure 2 compares six user-proxy LLMs across four datasets with a fixed judge (Claude-4-Sonnet) and assistant (GPT-4o). The left three columns of subplots report judge-based realism (GTEval, PI Δw, RNR; higher is better); the right three columns report human-anchored lexical diversity (MATTR, HD-D, Yule's K; best at the human z-score = 0). Dashed red (HH) and green (PP) lines provide calibration controls; error bars denote 95% CIs. We provide the assistant the reference chat history to anchor the rollout to the same trajectory as the original conversation; while LLM hallucinations can occasionally induce drift, a manual inspection found such divergence in mere $< 1\%$ of samples evaluated as part of 2.

Judge realism is consistent across datasets. Across GTEval, PI Δw, and RNR, *Gemini-2.5-Pro* and *Claude-4-Sonnet* are the most human-like on every dataset, with *GPT-4o* competitive but generally behind, and *GPT-OSS-120B* and *GPT-5* trailing. On ClariQ and QULAC, Claude-4-Sonnet/Gemini-2.5-Pro approach the HH ceiling under RNR, while PI Δw shows clear positive win margins for these models and negative/near-zero deltas for the rest. The agreement among the three judge metrics indicates a stable ordering not driven by any single rubric.

Diversity shows strong dataset effects and a realism–diversity tension. Lexical diversity diverges from the realism ranking. On *ClariQ*, most notably *Claude-4-Sonnet* and *GPT-5* exceed the human anchor on MATTR/HD-D and exhibit lower Yule's K, indicating a more diverse vocabulary than human questioners in this information-seeking regime. In contrast, *QULAC* exhibits a uniform diversity deficit: all proxies fall below the human baseline on MATTR and HD-D (with positive shifts in Yule's K), suggesting more templated clarifications than humans. *ChatbotArena* and *OASST1* sit between these extremes with smaller deviations around the human baseline. *Overall,* Gemini-2.5-Pro *&* GPT-4o *yield the strongest diversity alignment with humans across datasets, whereas*

Claude-4-Sonnet *generally tends to overshoot the human baseline. GPT-5* and *GPT-OSS-120B* have generally larger diversity difference compared to human baselines.

Judge realism and diversity are partially decoupled. High judge realism does not guarantee human-level diversity. *Claude-4-Sonnet* and *Gemini-2.5-Pro* lead on GTEval/PI/RNR but undershoot diversity on *QULAC*, indicating judges favor intent/style over surface variety. Conversely, GPT-4o shows more stable diversity with moderate judge-realism gains, underscoring that diversity alone is insufficient to achieve judge indistinguishability.

Uncertainty and robustness. Confidence intervals are generally narrow. When overlaps occur (e.g., GTEval on ChatbotArena), rank differences are small and concur with PI/RNR trends. For lexical-diversity panels, confidence intervals occasionally widen, notably for Yule's K, due to very short the user-side text (e.g., due to few turns) that increases estimator variance. Overall, Figure 2 supports three takeaways: (i) *Gemini-2.5-Pro* and *Claude-4-Sonnet* are reliably the most human-like by judge criteria; (ii) diversity depends strongly on dataset regime and proxies often lag on clarification-centric tasks like QULAC; and (iii) realism and diversity capture complementary facets of user-proxy quality, motivating the use of both families of metrics.

Specialized open-source proxy (USP-LLaMA). We additionally evaluate USP [36], a LLaMA-based model fine-tuned specifically for user simulation. Notably, despite being a smaller specialized model, USP achieves competitive realism scores, outperforming several much larger general-purpose LLMs on GTEval & RNR across all four datasets and on PI across two datasets, demonstrating that task-specific fine-tuning can be a cost-effective path toward human-like user simulation. On lexical diversity, USP produces richer vocabulary than real users on ChatbotArena, OASST1, and ClariQ, which suggests that the model has internalized varied language. The main limitation surfaces on QULAC, where PI deviation is notably high, suggesting that clarification-centric tasks remain challenging for specialized proxies. Overall, USP represents a promising direction: fine-tuned open-source proxies can rival proprietary frontier models on realism while being lightweight, though better calibration of lexical diversity and robustness across task types remain open challenges.

4.2 Judge Sensitivity Analysis

With the assistant and user-proxy fixed to GPT-4o, Figure 3 shows substantial judge sensitivity on realism scores. On GTEval, scores spread widely (≈ 0.45–0.81), with GPT-4o as judge yielding the highest value. PI is the most volatile: Gemini-2.5-Pro & Claude-4-Sonnet produce near-zero or negative win deltas, while GPT-5 & GPT-4o are clearly positive, suggesting family/self-preference or rubric alignment effects. RNR saturates near the ceiling for GPT-4o & GPT-5 judges (≈0.96–0.98) and is lower for others (≈0.79–0.82), indicating judge-dependent sensitivity: PI > GTEval > RNR.

These differences imply that conclusions drawn from a single judge can shift both the absolute level and the ordering of models. Since LLM judges can exhibit systematic biases [32, 35], they may behave either overly generous or overly conservative. Thus

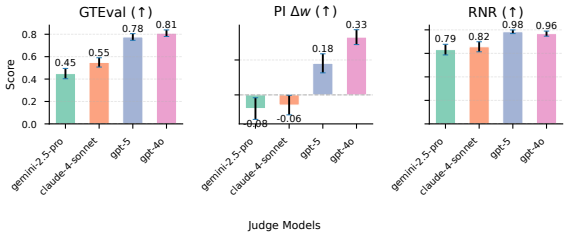

Figure 3: Judge sensitivity of judge-realism metrics on ChatbotArena with assistant & user-proxy both set to GPT-4o; bars vary the *judge* model. Error bars are 95% CIs.

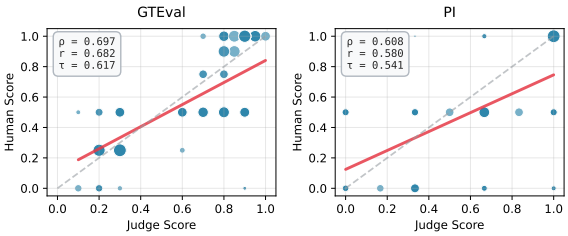

Figure 4: Judge–human correlation on ChatbotArena: Correlation of Claude-4-Sonnet judge scores & human scores for GTEval and PI, evaluated on *Gemini-2.5-Pro* user-proxy outputs (N=100 per metric). Solid line: linear fit; dashed: identity. Point size encodes local sample density. All correlations $p < 0.001$.

in practice, we recommend evaluating with *multiple judges* and applying HH/PP calibration when feasible, or normalizing per-judge before aggregation, to avoid over-interpreting judge-specific biases. Empirically, *Claude-4-Sonnet* behaves as a conservative yet stable judge, yielding non-saturated, well-separated scores across metrics, which is why we use Claude-4-Sonnet as a judge in the results reported in Figure 2.

4.3 Human-Judge Correlation

Although judge-sensitivity analysis (§4.2) helps characterize a judge's behavior on our task, it can still be difficult to interpret, since whether a judge appears "generous" or "conservative" is itself *task-dependent*. We therefore additionally assess judge reliability via human-judge correlation.

We validate judge reliability by correlating Claude-4-Sonnet judge scores with blinded human expert annotations on ChatbotArena. For the analysis shown in Figure 4, we stratified 100 episodes per metric (GTEval, PI) by judge score and conversation length, then compute Spearman's ρ [17], Pearson's r [19], and Kendall's τ [15] using human-judge label pairs. We observe that the GTEval aligns strongly with human judgments, while PI exhibits a moderate correlation. Note that PI is inherently harder to annotate because it requires pairwise comparisons; thus, a moderate correlation still indicates that PI is a reasonably reliable judge metric. Overall, these results suggest that judge scores track human perceptions of user-proxy quality.

To assess whether this finding generalizes, we also conduct a broader correlation study spanning three datasets and three proxy

models (50-100 human-annotated episodes per condition), shown in Appendix A. We observe GTEval ρ ranges from 0.607 to 0.697 and PI ρ from 0.532 to 0.671 (all $p < 0.001$), confirming that judge-human alignment is consistent regardless of dataset domain or proxy model.

4.4 Assistant Sensitivity and Rank Stability

LLM-as-judge scores can be confounded by the assistant model that the user proxy interacts with. To quantify this *assistant sensitivity*, we rerun evaluation on *ChatbotArena* while holding the judge fixed and varying the assistant among {GPT-4o, Claude-4-Sonnet, Gemini-2.5-Pro}. For each user-proxy model, we compute the three judge-based realism metrics (GTEval, PI, RNR) on the resulting rollouts.

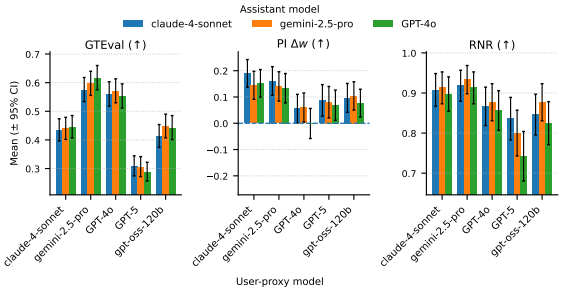

Figure 5: Assistant sensitivity on ChatbotArena (scores). Mean ± 95% CI of judge-based realism metrics for each user-proxy when swapping the *assistant model* (color); the judge is fixed to Claude-4-Sonnet.

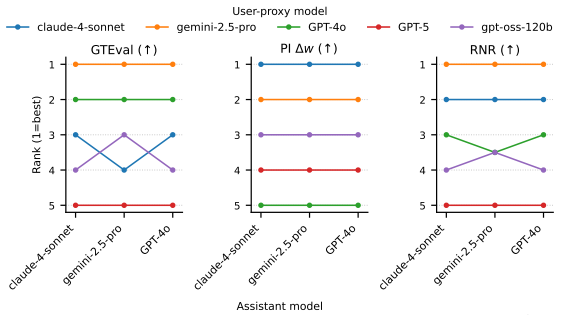

Figure 6: Rank stability under assistant swaps (ChatbotArena). Per-metric proxy ranks (1 = best) induced by the scores in Fig. 5 when varying the assistant model; stable lines indicate robust proxy ordering, while crossings reveal assistant-dependent rankings.

Figure 5 shows that absolute scores vary only slightly across assistants and are generally insufficient to change the overall ordering of user-proxy models. Figure 6 makes this explicit: ranks are stable (largely parallel trajectories) across assistants, with a single crossover in GTEval. Overall, assistant choice introduces mild variance in judge-based realism scores, but the resulting rankings remain stable most of the time.

4.5 Multi-Seed Robustness

LLM-as-judge metrics can fluctuate across repeated runs due to stochasticity in generation and evaluation. To assess the stability of our judge-based realism metrics, we perform a multi-seed analysis by repeating the benchmark five times with different random seeds for three different user-proxies while keeping the assistant and judge fixed.

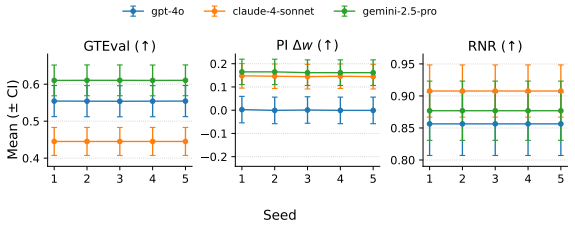

Figure 7: Multi-seed robustness of judge-based realism metrics. For each user-proxy model, points show the per-seed mean score and error bars indicate the corresponding 95% CI on ChatbotArena; the assistant (gpt-4o) and judge (claude-4-sonnet) are fixed across runs.

Figure 7 reports per-seed mean±95% CI for GTEval, PI Δw, and RNR. Across all three metrics, trajectories are nearly flat, indicating stability of judge-based metrics. The relative ordering of user-proxy models is consistent across seeds, and the small seed-to-seed fluctuations are dominated by within-run uncertainty captured by the error bars. Overall, these results suggest that our conclusions are robust to run-to-run variability in the rollout and judge scoring pipeline.

We also performed extended multi-seed analysis to all four datasets (3 seeds each for ClariQ, OASST1, Qulac), which can be found in Appendix A. Standard deviation remains ≤ 0.016 in all 12 conditions and proxy rankings are stable in 11 of 12 cases, with a single near-tie on OASST1 ($\Delta = 0.004$).

4.6 Goal-Conditioning Ablation

The goal description passed to each user proxy is synthesized by an auxiliary LLM from the reference conversation (see §3.1). To quantify how much this conditioning contributes to proxy performance and to assess whether the description leaks task content, we ran a three-condition ablation: `goal_full` (full 2–4 sentence intent and persona description, as used for the main experiments), `goal_topic` (a short 3-8 word topic phrase, e.g., "React `scrollIntoView` debugging"), and `goal_none` (no conditioning; equivalent to naive "act-as-a-user" prompting). We evaluate GPT-4o and Claude-4-Sonnet proxies on ChatbotArena and ClariQ datasets with a fixed Claude-4-Sonnet judge.

Table 2: Goal-conditioning ablation. GTEval and PI scores for GPT-4o / Claude-4-Sonnet proxies under three conditioning levels; judge fixed to Claude-4-Sonnet.

(a) ChatbotArena

Condition	GTEval	PI
goal_full	0.554 / 0.446	0.499 / 0.652
goal_topic	0.548 / 0.378	0.539 / 0.641
goal_none	0.156 / 0.018	0.296 / 0.022

(b) ClariQ

Condition	GTEval	PI
goal_full	0.228 / 0.213	0.419 / 0.620
goal_topic	0.219 / 0.189	0.591 / 0.838
goal_none	0.067 / 0.010	0.000 / 0.000

Table 2 showcases the results, where we observe two insightful findings. First, `goal_none` causes near-complete collapse on

both datasets: GTEval drops to near-zero and PI collapses to 0 on ClariQ, confirming that intent context is essential. Without it, proxies generate generic, repetitive phrasing with no anchor to a specific conversational objective, which judges immediately identify as non-human. Second, the PI/GTEval divergence under `goal_topic` is informative: PI rises *above* `goal_full` (e.g., ClariQ Claude: 0.838 vs. 0.620), while GTEval *drops* (ClariQ Claude: 0.189 vs. 0.213). A short topic phrase produces fluent, human-sounding turns, preserving surface-level indistinguishability, but without the full intent description the proxy drifts from the user's specific objective. This PI-GTEval divergence also serves as evidence against content leakage: if the full goal description were trivially encoding task structure, PI and GTEval would move together rather than in opposite directions. We provide detailed examples to illustrate this in Appendix B.

4.7 Telemetry, Scaling, and Cost

Based on our experiments, we further provide detailed analysis on the practical side of evaluation. Since most of the models we used are proprietary (incurs cost per million tokens), we provide statistical analysis on cost, latency and trade-off between cost vs performance. Per-sample telemetry (Fig. 8) shows that token usage is dominated by the judge, with the user-proxy & assistant contributing a smaller but non-negligible share; datasets differ markedly, *OASST1* drives the highest token counts while *ClariQ* yields the largest end-to-end latency.

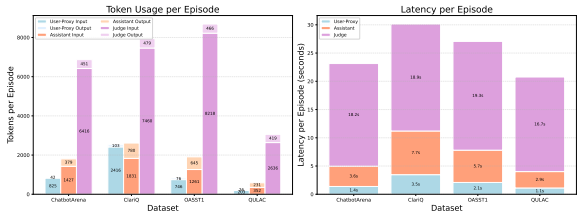

Figure 8: Avg per-sample telemetry for GPT-4o as user-proxy & assistant, and judged by Claude-4-Sonnet across 4 datasets on 6 metrics (Fig. 2). Left: Token usage by role (darker: input, lighter: output). Right: Cumulative latency per episode.

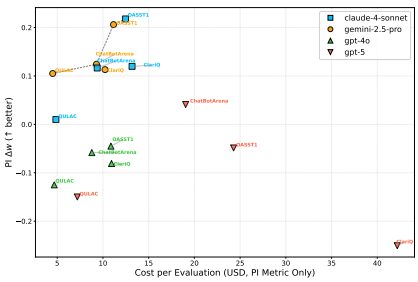

Figure 9: Cost-quality trade-off for PI: cost per evaluation (USD) vs. PI Δw (↑ better). Judge = Claude-4-Sonnet; assistant = GPT-4o; temperature = 0; cache off. Markers denote user-proxies; labels denote datasets. Dashed line: Pareto frontier. See Table 8 for model pricing.

The cost-quality frontier (Fig. 9) clarifies trade-offs for PI evaluation: User-proxies Gemini-2.5-Pro and Claude-4-Sonnet offer attractive Pareto points (good PI Δw at moderate cost), and GPT-5 generally incurs higher cost with weaker PI gain. Overall, user-proxy Gemini-2.5-Pro, along with Claude-4-Sonnet as judge, provides a balanced choice for large runs.

5 Conclusion & Future Work

MirrorBench provides a reproducible benchmark for evaluating the *human-likeness* of LLM-based user proxies, assessing how closely a proxy's utterances match real human users across conversational tasks, independent of downstream task success. Across four public datasets, we evaluate six user-proxy LLMs using three judge-based realism metrics (GTEval, PI, RNR) together with human-anchored lexical diversity measures (MATTR, HD-D, Yule's K). The resulting analyses highlight (i) systematic gaps between proxies and human users, (ii) a consistent realism-diversity trade-off that varies by dataset regime, and (iii) measurable sensitivity of absolute judge scores to the judge model, motivating calibration controls (HH/PP) and transparent multi-judge reporting. We also report token, latency, and cost profiles to make the evaluation trade-offs explicit when benchmarking at scale.

Limitations and outlook. Judge-based metrics may reflect model-family bias and prompt sensitivity; while HH/PP controls provide useful context, they do not eliminate all sources of bias. Our experiments primarily use a fixed assistant configuration and limited randomization, and our coverage is restricted to four English-centric datasets. Additionally, goal synthesis relies on an auxiliary LLM that produces behavioral abstractions of user intent; while this avoids verbatim leakage, the summarization process may smooth over atypical or incoherent user behavior, potentially causing the benchmark to underrepresent edge-case human interaction patterns. Furthermore, the current protocol evaluates proxies against a fixed reference trajectory by conditioning the assistant on the reference history; evaluating proxies in fully open-ended rollouts, where the assistant has no access to the reference requires a different evaluation methodology. Finally, lexical diversity captures surface-level distributional variation and does not fully represent interaction-level diversity such as repair, initiative, or long-horizon consistency.

Overall, MirrorBench is intended as a benchmark for systematically comparing user proxies and for grounding progress on realistic user simulation in conversational AI.

References

[1] Adnan Ahmad, Stefan Hillmann, and Sebastian Möller. 2025. Simulating User Diversity in Task-Oriented Dialogue Systems using Large Language Models. arXiv:2502.12813 [cs.CL] https://arxiv.org/abs/2502.12813

[2] Mohammad Aliannejadi, Julia Kiseleva, Aleksandr Chuklin, Jeff Dalton, and Mikhail Burtsev. 2021. Building and Evaluating Open-Domain Dialogue Corpora with Clarifying Questions. In *EMNLP*.

[3] Mohammad Aliannejadi, Hamed Zamani, Fabio Crestani, and W. Bruce Croft. 2019. Asking Clarifying Questions in Open-Domain Information-Seeking Conversations. In *Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval* (Paris, France) *(SIGIR'19)*. Association for Computing Machinery, New York, NY, USA, 475–484. doi:10.1145/3331184.3331265

[4] Anthropic. 2025. *System Card: Claude Opus 4 & Claude Sonnet 4.* Technical Report. Anthropic. https://www-cdn.anthropic.com/6d8a8055020700718b0c49369f60816ba2a7c285.pdf Accessed: 2025-10-22.

[5] Krisztian Balog and ChengXiang Zhai. 2023. User Simulation for Evaluating Information Access Systems. In *Proceedings of the Annual International ACM SIGIR Conference on Research and Development in Information Retrieval in the Asia Pacific Region* (Beijing, China) *(SIGIR-AP '23)*. Association for Computing Machinery, New York, NY, USA, 302–305. doi:10.1145/3624918.3629549

[6] Nicolas Bougie and Narimawa Watanabe. 2025. SimUSER: Simulating User Behavior with Large Language Models for Recommender System Evaluation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 6: Industry Track)*, Georg Rehm and Yunyao Li (Eds.). Association for Computational Linguistics, Vienna, Austria, 43–60. doi:10.18653/v1/2025.acl-industry.5

[7] Carlos P. Carmona, Robert Szava-Kovats, and Meelis Pärtel. 2019. Estimating probabilistic dark diversity based on the hypergeometric distribution. *bioRxiv* (2019). arXiv:https://www.biorxiv.org/content/early/2019/05/15/636753.full.pdf doi:10.1101/636753

[8] John W Chotlos. 1944. IV. A statistical and comparative analysis of individual written language samples. *Psychological Monographs* 56, 2 (1944), 75.

[9] Michael A. Covington and Joe D. McFall. 2010. Cutting the Gordian Knot: The Moving-Average Type–Token Ratio (MATTR). *Journal of Quantitative Linguistics* 17 (2010), 100 – 94. https://api.semanticscholar.org/CorpusID:18924254

[10] Yao Dou, Michel Galley, Baolin Peng, Chris Kedzie, Weixin Cai, Alan Ritter, Chris Quirk, Wei Xu, and Jianfeng Gao. 2025. SimulatorArena: Are User Simulators Reliable Proxies for Multi-Turn Evaluation of AI Assistants?. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng (Eds.). Association for Computational Linguistics, Suzhou, China, 35212–35290. doi:10.18653/v1/2025.emnlp-main.1786

[11] Google. 2025. *System Card: Gemini 2.5 Pro.* Technical Report. Google. https://storage.googleapis.com/deepmind-media/Model-Cards/Gemini-2-5-Pro-Model-Card.pdf Accessed: 2025-10-22.

[12] Ashutosh Hathidara, Julien Yu, and Sebastian Schreiber. 2025. Disambiguation-Centric Finetuning Makes Enterprise Tool-Calling LLMs More Realistic and Less Risky. arXiv:2507.03336 [cs.AI] https://arxiv.org/abs/2507.03336

[13] H. S. Heaps. 1978. *Information Retrieval: Computational and Theoretical Aspects.* Academic Press, Inc., USA.

[14] Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, et al. 2024. Gpt-4o system card. *arXiv preprint arXiv:2410.21276* (2024).

[15] M. G. Kendall. 1938. A NEW MEASURE OF RANK CORRELATION. *Biometrika* 30 (1938), 81–93. https://api.semanticscholar.org/CorpusID:120478295

[16] Seungone Kim, Jamin Shin, Yejin Cho, Joel Jang, Shayne Longpre, Hwaran Lee, Sangdoo Yun, Seongjin Shin, Sungdong Kim, James Thorne, and Minjoon Seo. 2024. Prometheus: Inducing Fine-Grained Evaluation Capability in Language Models. In *The Twelfth International Conference on Learning Representations*. https://openreview.net/forum?id=8euJaTveKw

[17] Stephen Kokoska and Dan Zwillinger. 2000. *CRC Standard Probability and Statistics Tables and Formulae, Student Edition.* doi:10.1201/b16923 Section 14.7.

[18] Andreas Köpf, Yannic Kilcher, Dimitri von Rütte, Sotiris Anagnostidis, Zhi Rui Tam, Keith Stevens, Abdullah Barhoum, Duc Minh Nguyen, Oliver Stanley, Richárd Nagyfi, Shahul ES, Sameer Suri, David Alexandrovich Glushkov, Arnav Varma Dantuluri, Andrew Maguire, Christoph Schuhmann, Huu Nguyen, and Alexander Julian Mattick. 2023. OpenAssistant Conversations - Democratizing Large Language Model Alignment. In *Thirty-seventh Conference on Neural Information Processing Systems Datasets and Benchmarks Track*. https://openreview.net/forum?id=VSJotgbPHF

[19] Charles J. Kowalski. 2018. On the Effects of Non-Normality on the Distribution of the Sample Product-Moment Correlation Coefficient. *Journal of the Royal Statistical Society Series C: Applied Statistics* 21, 1 (12 2018), 1–12. arXiv:https://academic.oup.com/jrsssc/article-pdf/21/1/1/48613051/jrsssc_21_1_1.pdf doi:10.2307/2346598

[20] Junyi Li, Xiaoxue Cheng, Xin Zhao, Jian-Yun Nie, and Ji-Rong Wen. 2023. HaluEval: A Large-Scale Hallucination Evaluation Benchmark for Large Language Models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 6449–6464. doi:10.18653/v1/2023.emnlp-main.397

[21] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. 2025. AgentBench: Evaluating LLMs as Agents. arXiv:2308.03688 [cs.AI] https://arxiv.org/abs/2308.03688

[22] Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. 2023. G-eval: NLG evaluation using gpt-4 with better human alignment. *arXiv preprint arXiv:2303.16634* (2023).

[23] Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. 2023. G-Eval: NLG Evaluation using Gpt-4 with Better Human Alignment. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 2511–2522. doi:10.18653/v1/2023.emnlp-main.153

[24] Jiarui Lu, Thomas Holleis, Yizhe Zhang, Bernhard Aumayer, Feng Nan, Haoping Bai, Shuang Ma, Shen Ma, Mengyu Li, Guoli Yin, Zirui Wang, and Ruoming Pang. 2025. ToolSandbox: A Stateful, Conversational, Interactive Evaluation Benchmark for LLM Tool Use Capabilities. In *Findings of the Association for Computational Linguistics: NAACL 2025*, Luis Chiruzzo, Alan Ritter, and Lu Wang (Eds.). Association for Computational Linguistics, Albuquerque, New Mexico, 1160–1183. doi:10.18653/v1/2025.findings-naacl.65

[25] Aengus Lynch, Benjamin Wright, Caleb Larson, Stuart J. Ritchie, Soren Mindermann, Evan Hubinger, Ethan Perez, and Kevin Troy. 2025. Agentic Misalignment: How LLMs Could Be Insider Threats. arXiv:2510.05179 [cs.CR] https://arxiv.org/abs/2510.05179

[26] Philip Mccarthy and Scott Jarvis. 2007. Vocd: A theoretical and empirical evaluation. Language Testing, 24, 459-488. *Language Testing - LANG TEST* 24 (10 2007), 459–488. doi:10.1177/0265532207080767

[27] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. 2018. Ray: a distributed framework for emerging AI applications. In *Proceedings of the 13th USENIX Conference on Operating Systems Design and Implementation* (Carlsbad, CA, USA) *(OSDI'18)*. USENIX Association, USA, 561–577.

[28] Tarek Naous, Philippe Laban, Wei Xu, and Jennifer Neville. 2025. Flipping the Dialogue: Training and Evaluating User Language Models. arXiv:2510.06552 [cs.CL] https://arxiv.org/abs/2510.06552

[29] OpenAI. 2025. *GPT-5 System Card.* Technical Report. OpenAI. https://cdn.openai.com/gpt-5-system-card.pdf Accessed: 2025-10-22.

[30] OpenAI. 2025. gpt-oss-120b & gpt-oss-20b Model Card. arXiv:2508.10925 [cs.CL] https://arxiv.org/abs/2508.10925

[31] Xuchen Pan, Dawei Gao, Yuexiang Xie, Yushuo Chen, Zhewei Wei, Yaliang Li, Bolin Ding, Ji-Rong Wen, and Jingren Zhou. 2024. Very Large-Scale Multi-Agent Simulation in AgentScope. arXiv:2407.17789 [cs.MA] https://arxiv.org/abs/2407.17789

[32] Aishwarya Sahoo, Jeevana Kruthi Karnuthala, Tushar Parmanand Budhwani, Pranchal Agarwal, Sankaran Vaidyanathan, Alexa Siu, Franck Dernoncourt, Jennifer Healey, Nedim Lipka, Ryan Rossi, Uttaran Bhattacharya, and Branislav Kveton. 2025. Quantitative LLM Judges. arXiv:2506.02945 [cs.CL] https://arxiv.org/abs/2506.02945

[33] J. Schatzmann and S. Young. 2009. The Hidden Agenda User Simulation Model. *Trans. Audio, Speech and Lang. Proc.* 17, 4 (May 2009), 733–747. doi:10.1109/TASL.2008.2012071

[34] Kumiko Tanaka-Ishii and Shunsuke Aihara. 2015. Computational constancy measures of texts-yule's k and rényi's entropy. *Comput. Linguist.* 41, 3 (Sept. 2015), 481–502. doi:10.1162/COLI_a_00228

[35] Aman Singh Thakur, Kartik Choudhary, Venkat Srinik Ramayapally, Sankaran Vaidyanathan, and Dieuwke Hupkes. 2025. Judging the Judges: Evaluating Alignment and Vulnerabilities in LLMs-as-Judges. In *Proceedings of the Fourth Workshop on Generation, Evaluation and Metrics (GEM²)*, Ofir Arviv, Miruna Clinciu, Kaustubh Dhole, Rotem Dror, Sebastian Gehrmann, Eliya Habba, Itay Itzhak, Simon Mille, Yotam Perlitz, Enrico Santus, João Sedoc, Michal Shmueli Scheuer, Gabriel Stanovsky, and Oyvind Tafjord (Eds.). Association for Computational Linguistics, Vienna, Austria and virtual meeting, 404–430. https://aclanthology.org/2025.gem-1.33/

[36] Kuang Wang, Xianfei Li, Shenghao Yang, Li Zhou, Feng Jiang, and Haizhou Li. 2025. Know You First and Be You Better: Modeling Human-Like User Simulators via Implicit Profiles. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, Vienna, Austria, 21082–21107. doi:10.18653/v1/2025.acl-long.1025

[37] Sai Wang, Senthilnathan Subramanian, Mudit Sahni, Praneeth Gone, Lingjie Meng, Xiaochen Wang, Nicolas Ferradas Bertoli, Tingxian Cheng, and Jun Xu. 2025. Configurable multi-agent framework for scalable and realistic testing of llm-based agents. arXiv:2507.14705 [cs.AI] https://arxiv.org/abs/2507.14705

[38] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. In *Proceedings of the 36th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) *(NIPS '22)*. Curran Associates Inc., Red Hook, NY, USA, Article 1800, 14 pages.

[39] Weinan Zhang, Muning Wen, Jun Wang, Haoyu Zhang, Qiuying Peng, Cheng Jin, Xihuai Wang, Qiqiang Lin, Xiaoyun Mo, and Jiamu Zhou. 2025. HammerBench: Fine-Grained Function-Calling Evaluation in Real Mobile Device Scenarios. arXiv:2412.16516 [cs.CL] https://arxiv.org/abs/2412.16516

[40] Yanzhe Zhang and Diyi Yang. 2025. Searching for Privacy Risks in LLM Agents via Simulation. arXiv:2508.10880 [cs.CR] https://arxiv.org/abs/2508.10880

[41] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-Judge with MT-Bench and

Chatbot Arena. In *Thirty-seventh Conference on Neural Information Processing Systems Datasets and Benchmarks Track*. https://openreview.net/forum?id=uccHPGDlao

[42] Xuhui Zhou, Zhe Su, Tiwalayo Eisape, Hyunwoo Kim, and Maarten Sap. 2024. Is this the real life? Is this just fantasy? The Misleading Success of Simulating Social Interactions With LLMs. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (Eds.). Association for Computational Linguistics, Miami, Florida, USA, 21692–21714. doi:10.18653/v1/2024.emnlp-main.1208

[43] Ruizhe Zhu, Hao Zhu, Yaxuan Li, Syang Zhou, Shijing Cai, Malgorzata Lazuka, and Elliott Ash. 2025. DialogueForge: LLM Simulation of Human-Chatbot Dialogue. arXiv:2507.15752 [cs.CL] https://arxiv.org/abs/2507.15752

A Extended Quantitative Results

Human-Judge Correlation Across Datasets and Proxies. Table 3 extends the human-judge correlation analysis from §4.3 beyond the single (Gemini-2.5-Pro, ChatbotArena) condition reported in the main paper. We provided blinded human annotators to score 50-100 episodes per condition across three datasets (ChatbotArena, OASST1, ClariQ) and three proxy models (Gemini-2.5-Pro, Claude-4-Sonnet, GPT-4o), then computed Spearman's ρ between human scores and Claude-4-Sonnet judge scores. GTEval ρ ranges from 0.607 to 0.697 and PI ρ from 0.532 to 0.671, all $p < 0.001$. The consistent alignment across diverse dataset domains and proxy models confirms that the judge reliability observed in the main paper is not specific to a single condition, and that Claude-4-Sonnet is a stable judge across the experimental configurations used in MirrorBench.

Table 3: Extended human-judge correlation. Spearman's ρ between Claude-4-Sonnet judge scores and blinded human annotations across four (proxy, dataset) conditions; all $p < 0.001$.

Dataset	Proxy	N	GTEval ρ	PI ρ
ChatbotArena	Gemini-2.5-Pro	100	0.697	0.608
OASST1	Claude-4-Sonnet	50	0.624	0.671
OASST1	GPT-4o	50	0.607	0.532
ClariQ	GPT-4o	50	0.686	0.596

Multi-Seed Robustness Across All Datasets. Table 4 extends the multi-seed robustness analysis from §4.5 beyond the 5-seed ChatbotArena runs shown in Figure 7. We ran 3 additional seeds on ClariQ, OASST1, and Qulac for three proxy models (GPT-4o, Claude-4-Sonnet, Gemini-2.5-Pro) with the assistant and judge fixed. GTEval mean ± SD is reported for all 12 conditions. SD is ≤ 0.016 in all cases, confirming that the low run-to-run variance observed on ChatbotArena generalizes across datasets. Proxy rankings are stable in 11 of 12 conditions; the single exception is OASST1 where Gemini-2.5-Pro and GPT-4o differ by only 0.004 (effectively tied). The canonical ordering Gemini-2.5-Pro > GPT-4o > Claude-4-Sonnet holds across all four datasets.

B Goal-Conditioning Ablation: Qualitative Examples

Table 5 illustrates the qualitative difference between `goal_full` and `goal_topic` conditioning on two representative ChatbotArena episodes. In each case, both conditions produce fluent, human-sounding turns (which is why PI stays high under `goal_topic`), but `goal_topic` causes the proxy to pursue a plausible *related* question rather than the user's actual intent (which is why GTEval drops).

In both examples, the `goal_full` proxy pursues the user's concrete objective (e.g., asking for a code usage example, staying on the same logical reasoning problem), while `goal_topic` produces a related but *generic* follow-up that a human asking the original question would not ask. This drift is captured by the drop in GTEval under `goal_topic` despite high PI, validating the interpretation in §4.6.

Table 4: Multi-seed GTEval stability (mean ± SD) across all four datasets. 5 seeds for ChatbotArena, 3 for the remaining datasets. SD ≤ 0.016 in all 12 conditions.

Proxy	ChatbotArena	ClariQ	OASST1	Qulac
GPT-4o	0.554 ± 0.000	0.239 ± 0.002	0.614 ± 0.000	0.351 ± 0.001
Claude-4-Sonnet	0.445 ± 0.000	0.210 ± 0.002	0.490 ± 0.007	0.288 ± 0.000
Gemini-2.5-Pro	0.610 ± 0.000	0.296 ± 0.007	0.618 ± 0.016	0.502 ± 0.013

C MirrorBench System

In this section, we explain the system aspects of the benchmarking framework. Although the metric definitions and execution may seem straightforward, substantial work went into creating a scalable, extensible system for flexible research use. We provide a detailed architectural blueprint in §C.1, explain the execution flow in §C.2, and provide metric implementation details in §C.3

C.1 Architecture

MirrorBench is a six-layer stack that cleanly separates infrastructure from evaluation logic (Fig. 10). Layers build upward, lower layers provide execution and data services; upper layers host pluggable components, task logic, and user interfaces.

C.1.1 Execution Backends & Persistence. This layer is responsible for all the execution and data management. The layer consists of two high-level modules explained below.

Execution Backend. It is responsible for taking decomposed units of tasks as input and send them for execution on the respective execution engine. The framework provides abstractions to support multiple execution backends to execute evaluation jobs at scale. The abstract interface provides uniform schema for implementations to extend the support for new execution engines. The framework supports fully-functional synchronous & asynchronous execution backends, and provides stub implementation to extend for Ray [27] backend.

Persistence. All evaluation artifacts are persisted in a SQLite database organized around a three-level hierarchy: *runs* define experiment configurations, *units* specify individual evaluation tasks (parameterized by user-proxy, dataset, metric, and seed), and *episodes* capture per-trial results including execution duration, metric values, and telemetry. Aggregate statistics (mean, standard deviation, confidence intervals) are materialized in a separate `metrics` table to support efficient querying, while composite scores across multiple metrics are stored in `scorecards` for holistic comparisons. The complete schema is detailed in Table 6.

Table 5: Qualitative comparison of `goal_full` vs. `goal_topic` conditioning on two ChatbotArena episodes (GPT-4o proxy). `goal_full` stays on the user's specific request; `goal_topic` deflects to a generic related question.

Episode	Real user turn	Proxy turn (`goal_full`)	Proxy turn (`goal_topic`)
JavaScript ArrayBuffer	*"Please write a JavaScript function that receives an ArrayBuffer, parses it and returns it as a JavaScript object."*	"Can you include an example of how to **use the function** with a sample ArrayBuffer?"	"Can you explain how to **convert an ArrayBuffer to a string** in JavaScript?"
Speed comparison	*"mark is faster than mary. mary is faster than joe. is joe faster than mark?"*	"If Mark is faster than Mary, and Mary is faster than Joe, does that mean **Mark is also faster than Joe**?"	"Who is faster, **Usain Bolt or a cheetah**?"

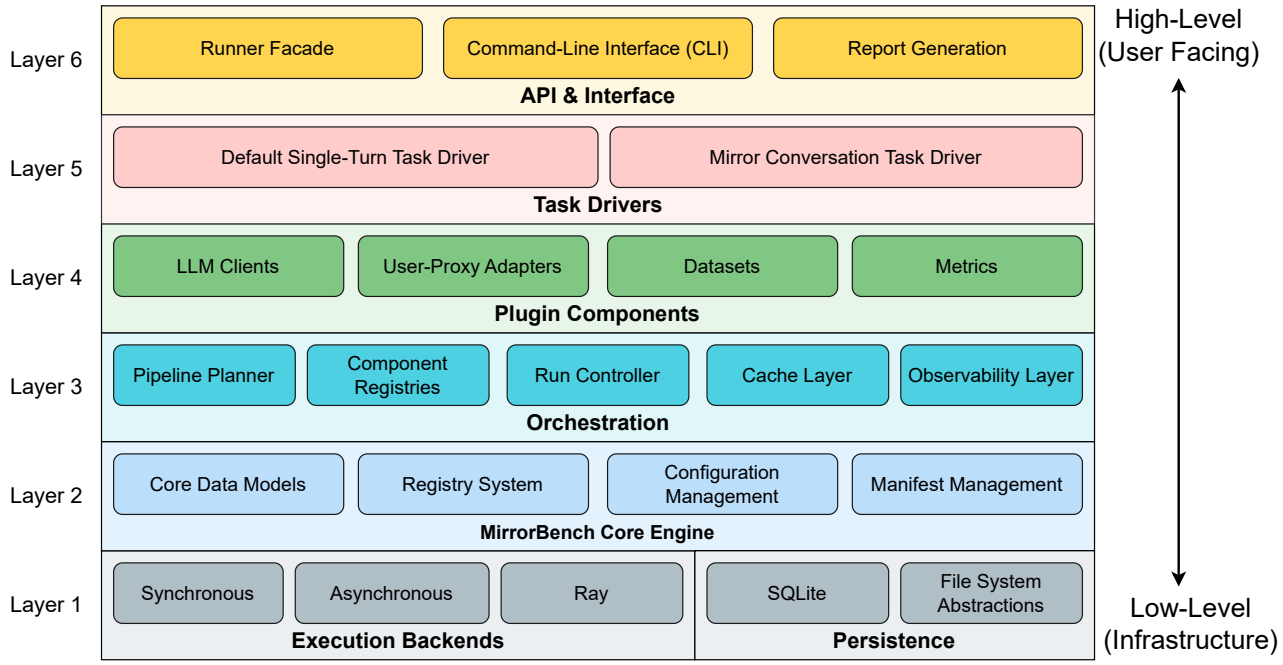

Figure 10: MirrorBench Architecture: Six-layer stack from low-level execution backends & persistence up through the core engine, plugin components, CLI & reporting, and task drivers. Top layers are user-facing; bottom layers are low-level infrastructure abstractions.

C.1.2 Core Engine. All the core functionalities like data models, registry builder, configuration and manifest management are contained in Layer 2 of the architecture. Since Layer 1 provides uniform abstractions to produce and consume objects, this layer considers backend-agnostic implementation.

Data Models. All the data structures for messages, episodes, evaluation units and run manifests are defined with typed data models using `TypedDict` and `pydantic`. We formally explain the context of these different data objects later.

Registry Builder. The framework provides certain high-level components which requires module-specific registries. This layer provides the abstract registry builder that supports creating metadata-aware registry for the downstream modules. Such registries can avoid silent errors since we can ensure certain constraints on the classes being registered with the metadata.

Configuration & Manifest Management. A typical evaluation job requires configuring user-proxy, datasets, metrics, scorecards etc. We define standard configuration schema for each of such components. Moreover, upon invocation of job or during dryrun, the system decomposes evaluation task into small evaluation units. In order to support parallelization (for async and remote backends), we persist self-contained information about the executable units as manifest files.

C.1.3 Orchestration. This layer coordinates the evaluation workflow by managing component lifecycles, execution planning, and runtime operations. The layer consists of five orchestration modules.

Pipeline Planner. It validates job configurations and generates execution plans by decomposing evaluation jobs into granular evaluation units (user-proxy, dataset, metric, seed combinations). The

Table 6: Complete MirrorBench Run Database Schema

Table	Field	Type	Description
runs	run_id	TEXT	Unique identifier for the experiment run (primary key)
	created_at	TEXT	ISO-8601 timestamp of run creation
	status	TEXT	Run status: `created`, `running`, `completed`, `failed`
	engine	TEXT	Execution engine identifier (e.g., `sync`, `async`)
	planner_version	TEXT	Version of the planner used for the run
	summary_json	TEXT	JSON-encoded run-level summary statistics
	notes	TEXT	User-provided annotations or metadata
units	run_id	TEXT	Foreign key to parent run
	unit_id	TEXT	Unique identifier for the evaluation unit (composite primary key)
	user_proxy	TEXT	User-proxy agent implementation identifier
	dataset	TEXT	Dataset name (e.g., `oasst1`, `clariq`)
	metric	TEXT	Evaluation metric (e.g., `gteval`)
	seed	INTEGER	Random seed for reproducibility
	judge	TEXT	Optional judge model for LLM-based metrics
	status	TEXT	Unit execution status: `pending`, `running`, `completed`
episodes	run_id	TEXT	Foreign key to parent run
	unit_id	TEXT	Foreign key to parent unit
	episode_id	TEXT	Unique episode identifier (primary key)
	status	TEXT	Episode outcome: `success`, `failure`, `timeout`
	duration_s	REAL	Wall-clock execution time in seconds
	artifact_path	TEXT	File path to serialized conversation artifact
	summary	TEXT	Human-readable episode summary
	metric_values	TEXT	JSON-encoded per-metric scores for this episode
	telemetry_json	TEXT	JSON-encoded execution telemetry (token counts, API calls)
metrics	run_id	TEXT	Foreign key to parent run
	unit_id	TEXT	Foreign key to parent unit
	metric	TEXT	Registered metric name (primary key)
	mean	REAL	Sample mean across episodes
	standard_deviation	REAL	Sample standard deviation
	confidence_interval	REAL	95% confidence interval half-width
	p_value	REAL	Statistical significance (e.g., vs. baseline)
	sample_size	INTEGER	Number of valid episodes
	extras	TEXT	JSON-encoded additional statistics
telemetry	run_id	TEXT	Foreign key to parent run
	unit_id	TEXT	Foreign key to parent unit
	key	TEXT	Telemetry key (e.g., `total_tokens`, `api_errors`)
	value	TEXT	Aggregated telemetry value (JSON or scalar)
scorecards	run_id	TEXT	Foreign key to parent run
	name	TEXT	Scorecard identifier (e.g., `realism`, `diversity`)
	score	REAL	Weighted composite score
	weights	TEXT	JSON-encoded metric weights used for aggregation
	missing_metrics	TEXT	JSON list of metrics excluded due to missing data
	extras	TEXT	JSON-encoded additional scorecard metadata

planner performs compatibility filtering based on component metadata to ensure valid combinations and produces reproducible plan manifests.

Component Registries. It maintains metadata-aware registries for all pluggable components including user-proxy adapters, datasets, metrics, model clients, and judges. Registries support dynamic component discovery and instantiation through decorator-based registration patterns.

Run Controller. It orchestrates end-to-end evaluation runs by managing execution flow, coordinating backend selection (synchronous, asynchronous, or Ray), tracking progress, and handling run lifecycle including resumption of interrupted runs.

Cache Layer. It provides SQLite-based caching for model responses to reduce redundant API calls and improve evaluation efficiency. The cache uses content-based keys (hashing model name,

messages, and configuration) and supports namespace isolation and TTL-based expiration.

Observability Layer. It provides structured logging via `structlog` and optional OpenTelemetry integration for distributed tracing and metrics collection. The layer supports multiple output formats (JSON, text) and configurable log levels for debugging and monitoring.

C.1.4 Plugin Components. This layer contains the core pluggable components that implement domain-specific evaluation logic. All components follow standardized interfaces and register through the registry system.

Model Clients. It wraps provider SDKs (OpenAI, LangChain, Azure, etc.) with a unified interface for LLM invocation. Clients automatically capture telemetry (tokens, latency, cost) and support transparent caching through wrapper composition.

User-Proxy Adapters. It adapts diverse agent frameworks and LLM configurations into a standardized `AgentSession` interface. Adapters normalize differences in message formats, tool usage patterns, and execution models across frameworks.

Datasets. It loads and normalizes evaluation datasets from various sources (HuggingFace, JSONL files, etc.) into standardized episode objects. Loaders support dataset splitting, sampling limits, and provide metadata about available splits and reference statistics.

Metrics. It implements human-likeness measurements including lexical diversity metrics (MATTR, HD-D, Yule's K) and LLM-as-judge metrics (GTEval, PI, RNR). Metrics declare compatibility requirements through metadata (task types, judge dependencies, reference statistics needs).

C.1.5 Task Drivers. This layer implements interaction protocols that orchestrate how the artificial conversation between user-proxy & assistant agent is synthesized and how it is engaged with the evaluation metrics. Task drivers bridge the gap between high-level evaluation specifications and low-level execution.

Default Single-Turn Task Driver. It executes single-turn interactions where the user-proxy receives episode context and generates a single response. This driver is suitable for simple prompt-completion tasks and turn-level utterance evaluation.

Mirror Conversation Task Driver. It orchestrates multi-turn conversational interactions by executing sequences of user-proxy responses within episodic contexts. This driver supports conversation-level evaluations where human-likeness is assessed across extended interactions.

C.1.6 API & Interface. This layer provides user-facing interfaces for interacting with the framework through programmatic APIs and command-line tools.

Runner Facade. It provides a simplified programmatic API for executing evaluation runs from Python code. The facade abstracts orchestration complexity and exposes high-level methods for planning, execution, and result retrieval with progress tracking.

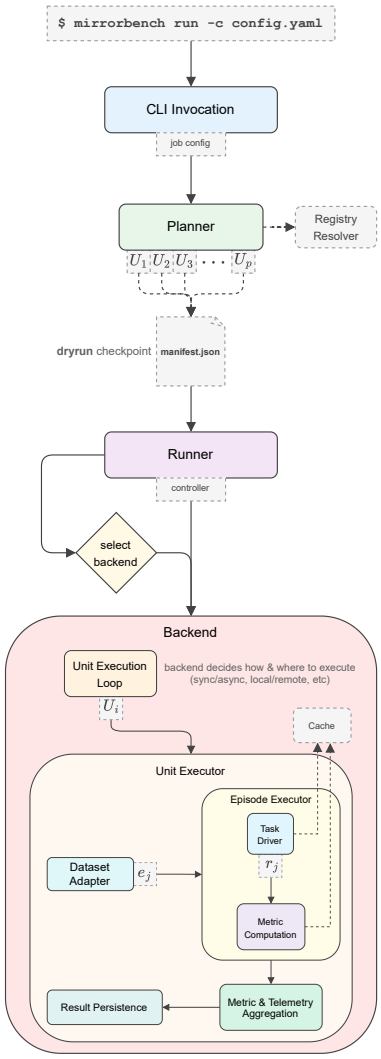

Figure 11: MirrorBench execution flow. The framework decomposes an evaluation job into *units* $\{U_1, \ldots, U_p\}$; each unit U_i iterates over *episodes* $\{e_1, \ldots, e_n\}$ produced by the dataset adapter and executed via task drivers. Metrics are computed per episode and aggregated within each unit with confidence intervals.

Command-Line Interface (CLI). It exposes framework functionality through a comprehensive CLI supporting workflow commands (`plan`, `dryrun`, `run`), reporting (`report json`), run management (`runs inspect`, `runs delete`), and cache operations (`cache stats`, `cache purge`).

Report Generation. It produces structured evaluation summaries with aggregated statistics including means, confidence intervals, and significance testing. The module supports JSON report output with planned support for HTML visualization.

C.2 Execution Flow

This section details the end-to-end execution that instantiates the pipeline Ψ_T (as described in §3.1). Figure 11 shows the path from CLI invocation through planning, orchestration, and nested execution to persisted results, compiling high-level specifications into reproducible evaluation artifacts.

CLI Invocation and Planning. An evaluation job is declared via a configuration file (appendix §D.2) that specifies user proxies $\mathcal{P} = \{\theta_{u_1}, \ldots, \theta_{u_q}\}$, datasets $\mathcal{D} = \{D_1, \ldots, D_d\}$, the metric set $M = \{m_1, \ldots, m_k\}$, a fixed assistant model θ_a, and run parameters (backend, observability, maximum concurrency, and a seed set $S = \{s_1, \ldots, s_r\}$). The seed set enables repeated trials of the same combination under distinct random initializations, supporting variance estimation and confidence intervals.

The *Planner* validates the configuration against component registries and enumerates all mutually compatible tuples to produce execution units $U = \{U_1, \ldots, U_p\}$.

$$U = \left\{ (\theta_{u_x}, D_y, m_z, s_w) \;\middle|\; \begin{array}{l} \theta_{u_x} \in \mathcal{P},\ D_y \in \mathcal{D}, \\ m_z \in M,\ s_w \in S, \\ \mathrm{Compat}(\theta_{u_x}, D_y, m_z) \end{array} \right\}$$

Here Compat$(\cdot)$ returns `true` iff the components can be jointly executed under their declared capabilities and requirements (e.g. metric prerequisites, model/back-end constraints). Each unit $U_i = (\theta_{u_x}, D_y, m_z, s_w)$ is executed and evaluated independently, with models, prompts, and driver assignments fixed at plan time. The planner creates & persists a replayable *manifest* file (appendix §D.2) capturing all requisite metadata, and exposes a `dryrun` checkpoint for inspection prior to committing computational resources.

Orchestration and Backend Selection. The *Runner* instantiates a *Run Controller* that manages execution state, telemetry, and result persistence. The controller initializes a run-specific SQLite database to store task driver outputs and metric values. Based on the run configuration, the system selects an available backend (sync, async, or distributed) and dispatches the list of units U for execution under the chosen strategy. For each unit $U_i \in U$, the backend invokes a unit executor that loads the dataset, instantiates the proxy, initializes the metrics, and configures the task driver.

Unit Execution and Episode Extraction. For each unit $U_i = (\theta_{u_x}, D_y, m_z, s_w)$, the unit executor loads dataset D_y and enumerates its constituent episodes. Each episode e_j is an immutable specification of a single reference dialogue (§3.1), with dataset layout and fields given by

$$\begin{aligned} D_y &= \{\, e_1, e_2, \ldots, e_n \,\}, \\ e_j &= \big(h_j^{\mathrm{ref}},\ g_j,\ \mathrm{meta}_j\big), \\ h_j^{\mathrm{ref}} &= \big[(u_{j,1}^{\mathrm{ref}}, a_{j,1}^{\mathrm{ref}}), \ldots, (u_{j,L_j}^{\mathrm{ref}}, a_{j,L_j}^{\mathrm{ref}})\big], \end{aligned}$$

where g_j denotes the user goal (explicit or inferred), and meta_j is an optional field containing evaluation-specific annotations (e.g., task tags or reference statistics). The unit executor iterates over the episode set $\{e_j\}$, delegating each episode to the appropriate episode executor for execution. Episode executor first synthesizes proxy-assistant conversation using task driver and later compute evaluation results using metric implementation m_z.

Conversation Synthesis via Task Drivers. For each episode e_j, the task driver orchestrates the synthesis of a proxy-assistant interaction by alternating invocations of the user proxy θ_{u_x} (abbrev. θ_u) and the assistant model θ_a, matching the reference dialogue length $L_j = |h_j^{\mathrm{ref}}|$. Let $\hat{h}_{j,t}$ denote the synthetic history after t turns, with $\hat{h}_{j,0} = \langle\,\rangle$. The synthetic rollout for episode e_j is:

Initialize: $\hat{h}_{j,0} \leftarrow \langle\,\rangle$

For $t = 1, \ldots, L_j$**:**

$$\begin{aligned} \hat{u}_{j,t} &\leftarrow \theta_u\big(\hat{h}_{j,t-1},\ g_j\big), \\ \hat{a}_{j,t} &\leftarrow \theta_a\big(\hat{h}_{j,t-1} \parallel (\hat{u}_{j,t}),\ h_j^{\mathrm{ref}}\big), \\ \hat{h}_{j,t} &\leftarrow \hat{h}_{j,t-1} \parallel (\hat{u}_{j,t}, \hat{a}_{j,t}). \end{aligned}$$

The driver records per-turn telemetry (latency and token usage) and produces the *episode artifact*

$$r_j = \big(\hat{h}_{j,L_j},\ e_j,\ \mathrm{telemetry}_j\big),$$

which binds the synthetic transcript to its reference. The synthetic history is persisted in cache, ensuring that other metric computations can reuse it.

Metric Computation. After producing an episode artifact r_j, metric m_z evaluates r_j to yield a scalar value v_{j,m_z}. Depending on the metric type (e.g., lexical, or LLM-judge), the computation may involve reference comparison, pairwise scoring, or external model inference. Formally,

$$v_{j,m_z} = m_z\big(r_j\big),$$

where $v_{j,m_z} \in \mathbb{R}$ represents the realism score for metric m_z on episode e_j. Each evaluation may also produce auxiliary data such as judge verdicts, raw reasoning traces, or calibration diagnostics. The full episode artifact, together with all metric outputs, is persisted to disk for aggregation, post-hoc inspection and reproducibility.

Result Aggregation. After processing all episodes $\{e_1, \ldots, e_n\}$ within unit U_i, the executor aggregates the metric outputs into summary statistics. For metric m_z, let $V_{U_i,m_z} = \{v_{1,m_z}, v_{2,m_z}, \ldots, v_{n,m_z}\}$ denote the set of episode-level scores. The unit-level aggregate includes the mean, standard deviation, and 95% confidence interval:

$$\begin{aligned} \bar{v}_{U_i,m_z} &= \frac{1}{n}\sum_{j=1}^{n} v_{j,m_z}, \\ s_{U_i,m_z} &= \sqrt{\frac{1}{n}\sum_{j=1}^{n}(v_{j,m_z} - \bar{v}_{U_i,m_z})^2}, \\ \mathrm{CI}_{95}(U_i, m_z) &= \begin{bmatrix} \bar{v}_{U_i,m_z} - 1.96\,\dfrac{s_{U_i,m_z}}{\sqrt{n}} \\ \bar{v}_{U_i,m_z} + 1.96\,\dfrac{s_{U_i,m_z}}{\sqrt{n}} \end{bmatrix}. \end{aligned}$$

The run database stores these aggregates, telemetry totals, and per-episode status indicators (success/failure).

Iteration and Reporting. The backend executes each unit $U_i = (\theta_{u_x}, D_y, m_z, s_w)$ in $U = \{U_1, \ldots, U_p\}$. Upon completion, the controller updates the last-run pointer for convenient access and consolidates a unified `RunSummary` that collates, for every m_z and unit U_i, the tuple $\big(\bar{v}_{U_i,m_z},\ s_{U_i,m_z},\ \mathrm{CI}_{95}(U_i, m_z)\big)$, along with cumulative telemetry and execution metadata. This summary serves as the

Table 7: Experimental Configuration and Dependency Versions

Category	Component	Version/Configuration
Hardware & Infrastructure		
OS	Linux	SUSE Linux Enterprise Server 15 SP4 (kernel 5.14+)
CPU	Standard	Intel Xeon Platinum 8468 (16+ cores)
RAM	Standard	64 GB DDR4
GPU	GPT-OSS-120B only	4× NVIDIA H200 (140GB VRAM each) or equivalent
CUDA	GPT-OSS-120B only	CUDA 12.4+
Runtime & Storage		
Python	Interpreter	3.12.0+
SQLite	Database	3.45.0+ (WAL mode enabled)
API Providers (accessed Oct–Dec 2024)		
OpenAI	GPT-4o	`gpt-4o-2024-08-06`
	GPT-5	`gpt-5-2025-08-07`
Anthropic	Claude-4-Sonnet	`claude-sonnet-4-20250514`
Vertex AI	Gemini-2.5-Pro	`2025-06-17`
Self-hosted	GPT-OSS-120B	vLLM 0.10.1+gptoss, BF16 precision
Core Library Versions		
Model Clients	`openai`	1.40.0+
	`langchain-openai`	0.1.0+
Data	`pydantic`	2.5.0+
	`datasets`	4.1.1+ (HuggingFace)
Utilities	`tiktoken`	0.7.0+ (tokenization)
	`nltk`	3.9.2+ (sentence segmentation)
	`tenacity`	8.2.3+ (retry logic)
Logging	`structlog`	24.1.0+
Telemetry	`opentelemetry-sdk`	1.37.0+
	`opentelemetry-exporter-otlp`	1.37.0+

canonical artifact for subsequent JSON/HTML report generation (appendix §D.2) and cross-proxy comparison. The caching layer eliminates repeated LLM calls across units/runs, and the plan manifest enables exact replays of component instantiation and invocation for reproducible results.

C.3 Implementation Details of Metrics

Below, we provide the implementation details for the metrics and other associated components in the evaluation pipeline. Additionally, in Table 7, we provide details of the machine we used to run all the experiments mentioned in this paper.

Tokenization. All lexical metrics use the GPT-4o tokenizer (via `tiktoken`) to ensure consistency with modern LLM vocabularies. User utterances in the dialogue sample are concatenated with space separators before tokenization.

Judge Caching. Judge responses are cached using content-based keys (hash of model name, messages, temperature, and metric parameters) with namespace isolation per metric. Default TTL is 30 days. This enables reproducible experiments and cost reduction in iterative evaluation.

Multiple Judge Calls. By default, each judge metric runs c independent samples per episode (c is a metric parameter to be set) with different random seeds (for PI's order randomization). Final scores are averaged; individual samples and reasoning traces are stored in metadata for audit.

Statistical Reporting. Aggregated metrics report mean, standard deviation, and 95% confidence intervals using Student's t-distribution (Equation 4). The framework optionally supports bootstrap confidence intervals (1000 iterations) for non-parametric estimation.

Aggregation Across Episodes. For each metric M and proxy-dataset pair, the framework computes per-episode scores $\{s_1, s_2, \ldots, s_n\}$ and aggregates them using mean, standard deviation, and confidence intervals. For lexical metrics, z-scores are computed per-episode using the human baseline statistics, then aggregated. For judge metrics, raw scores are averaged first; if calibration is enabled, HH and PP statistics are computed separately and applied to normalize the aggregated results.

Handling Missing References. Episodes missing human reference conversations (required for GTEval, PI, and z-score computation) are excluded from metric computation. The framework logs these exclusions and reports the count of valid episodes in the metadata. This ensures that all reported statistics are based on complete, valid comparisons.

Stability Filters. Lexical metrics require a minimum token count (default: 5) to produce stable estimates. Episodes below this

Table 8: Model Pricing (USD per million tokens). Pricing as of 25 October 2025 from the provider's public APIs.

Model	Input (prompt)	Output (completion)
GPT-4o	$2.50	$10.00
GPT-5	$1.25	$10.00
Claude 4 Sonnet	$3.00	$15.00
Gemini 2.5 Pro	$1.25	$10.00

threshold are flagged as unstable and optionally excluded from aggregation. This prevents outlier scores from very short utterances (e.g., single-word responses) from skewing results.

LLM Settings. All the LLMs used during the evaluation have fixed `temperature=0` and `max_tokens=2048`. Moreover, all the LLM clients have by default *"retry with exponential backoff"* scheme enabled where `backoff_duration=2s` and `max_retries=5`.

Telemetry & Observability. The framework captures per-invocation metrics (latency-to-first-token, generation time, token counts, cost) and organizes traces into hierarchical spans (`run`, `unit`, `episode`) annotated with run/unit identifiers, status, and dataset metadata. Structured logs use `structlog` with JSON formatting and contextual bindings.

Model Pricing. Cost estimates reported in our experiments are computed using the provider-published pricing as of 25 October 2025, shown in Table 8. All costs are calculated in USD per million tokens for both input (prompt) and output (completion) tokens. Token counts are obtained directly from provider API responses. The total cost C_{USD} for each episode is computed as

$$C_{USD} = \frac{\text{tokens}_{\text{input}} \times \text{price}_{\text{input}} + \text{tokens}_{\text{output}} \times \text{price}_{\text{output}}}{10^6}$$

Later, we aggregate across all invocations within a unit. Note that pricing may vary by region, deployment tier, or provider discounts; we report costs using standard public API rates.

D Prompts, Config, Manifest & Report

This appendix provides the exact artifacts required to reproduce our runs: judge prompts, task-driver prompts, job configuration, and the emitted plan manifest. For reproducibility, we report the prompt, model identifiers, and key decoding parameters (temperature, max tokens).

D.1 System Prompts

We provide system prompts used across the evaluation pipeline (metric judges, user-proxy, assistant).

PI Judge Prompt

[System]
Please act as an impartial judge and choose the most realistic USER among the two conversations provided below. You should choose the conversation which has USER turns written by a real user in your opinion. Based on your experience & knowledge of human conversation, determine which USER in which conversation sounds more like a real human interacting with a chatbot. Note that your judgement should be based on style, tone and behavior of USER rather than the response quality of their utterances. Avoid any position biases and ensure that the order in which the responses were presented does not influence your decision. Do not allow the length of the USER messages to influence your evaluation. Do not let ASSISTANT messages influence your decision. Your evaluation must only be dependent on USER messages only. Be as objective as possible.

Before giving your final evaluation verdict, you should first think through and produce your reasoning step-by-step. Then, based on your reasoning, give your final verdict on which conversation has USER turns that sound more like a real human interacting with a chatbot.

If Conversation A is more real, then verdict would be "A". If Conversation B is more real, then verdict would be "B". If both are equally real or non-real, then verdict would be "Tie".

Your output must be of following JSON format: {"reasoning": "<short reason>", "verdict": "A"|"B"|"Tie"}.

[Conversation A]
{conversation_a}

[Conversation B]
{conversation_b}

Output ONLY valid JSON, no additional text.

Figure 12: Judge prompt for Pairwise Indistinguishability Metric

GTEval Judge Prompt

You are an expert evaluator of conversational AI systems. Your task is to compare two conversations and assess whether the user responses in the proxy conversation exhibit similar style, tone, and realism to the user responses in the real conversation.

Real Conversation:
{real_conversation}

Proxy Conversation (where user turns are generated by user proxy agent):
{proxy_conversation}

Evaluation Criteria:

1. **Style Similarity**: Do the proxy user responses match the conversational style of real user responses (formality, tone, verbosity)?
2. **Realism**: Do the proxy user responses sound natural and human-like?
3. **Contextual Appropriateness**: Are the proxy user responses appropriate given the conversation context?
Note: You should not evaluate based on the content of the responses, only their style, realism, contextual appropriateness, and tone.

Instructions:
- Focus exclusively on comparing USER responses (ignore assistant responses)
- Consider the overall conversation flow, not just individual turns
- Evaluate style, tone, and naturalness without encouraging unsafe behaviour
- If the comparison cannot be performed safely, explain the reason and provide a score of 0.0
- Otherwise, provide detailed reasoning about the comparison and assign a score from 0.0 to 1.0 where:
- 0.0 = Proxy user responses are completely unlike real user responses
- 0.5 = Proxy user responses are somewhat similar but have noticeable differences
- 1.0 = Proxy user responses are highly similar in style and realism

Provide your evaluation as a JSON object with the following structure:
{ "reasoning": "Detailed explanation of your evaluation (2-3 sentences)", "score": <float between 0.0 and 1.0> }

Output ONLY valid JSON, no additional text.

Figure 13: Judge prompt for GTEval Metric

RNR Judge Prompt

[System]
Please act as an impartial judge and provide verdict on the realism of the USER turns in the conversation provided below based on the provided rubric. You should avoid checking the responses of the ASSISTANT messages since they are not relevant to the task. Your evaluation should follow the rubric provided. Note that your judgement should be based on style, tone and behavior of USER rather than the response quality of their utterances.

[Rubric for scoring USER realism]
Provide verdict on how human-like the USER is on ("NO" or "YES") using this rubric (verdict "NO" means not real, verdict "YES" means real):
1. Concise and real-user like language
2. Does not sound scripted or artificial
3. Real-user like tone and style

Return JSON: {"reasoning": "<1-2 sentences>", "verdict": <"NO" or "YES">}.

[Conversation]
{conversation}

Output ONLY valid JSON, no additional text.

Figure 14: Judge prompt for RNR Metric

User-Proxy System Prompt

You are simulating a real human user for the MirrorBench evaluation harness. Respond with the next USER turn only. Do not write assistant messages, notes, or any other analysis. Your utterance should be like a real user and the context should be based on the following information provided.

Task description: {task_description}

Match the length, tone, and specificity of real user utterances. If you are unsure, respond naturally based on the assistant's previous messages like how a real human would. Note that your response MUST not contain anything other than the USER utterance. Do not include any prefixes like 'User:' or 'Human:' as well. Just the raw message content.

Figure 15: System prompt for User-Proxy

Assistant System Prompt

You are the assistant in a MirrorBench replay. The user-proxy agent is attempting to reproduce the USER side of the real conversation provided below. But the user-proxy does not have access to the real conversation history. Instead, it only has access to the conversation summary.

You need to respond as the assistant. But we are providing you with the real conversation history as context, so you can respond consistently same as (or similar to) the original assistant in the real conversation (you may paraphrase lightly for safety).

If user-proxy deviates from the original USER turn or the original response would violate policy, reply helpfully using your own knowledge while remaining consistent with the persona demonstrated so far. Always follow ethical content policies. Paraphrase sensitive content instead of quoting it verbatim, and refuse politely if a

request is disallowed.

Here is the real conversation for context (the USER turns are from the original conversation):
{real_conversation}

Now, we will provide you with ongoing conversation with the user-proxy. Please respond as the assistant in this conversation.

Figure 16: System prompt for Assistant

Figure 15 and 16 show system prompts for user-proxy and assistant respectively. Note that once filled, the system prompt would go into chat history as system message. Thus, we do not see the chat history-related placeholder in these system prompts.

D.2 Evaluation Config, Manifest & Report

Figure 19 shows the example job configuration. The configuration is declarative: the `run` block fixes execution semantics (backend, concurrency, timeouts), reproducibility (`seeds`), and infrastructure knobs (cache and logging). In our example, the async backend with `max_concurrency=8`, while caching is enabled to deduplicate repeated LLM calls across units. The `user_proxies` block registers a concrete proxy via a lightweight adapter and a model client (here, LangChain's `ChatOpenAI` wrapper for GPT-4o), keeping proxy identity and invocation details separate from orchestration. The `datasets` block binds the ChatbotArena's MirrorBench corresponding split to a local JSONL path and caps evaluation at 200 examples for controlled runtime.

Metrics are declared under `metrics` with their own clients and prompts shown in D.1, allowing the judge to differ from the proxy/assistant. We show two metrics, GTEval and Pairwise Indistinguishability, both routed to Claude-4-Sonnet at `temperature=0`. Each metric includes `num_judge_samples` (for repeated scoring) and `compute_controls=true`, which triggers the HH/PP control comparisons used for calibration and bias checks. Finally, the task drivers section binds the dataset to the Mirror Conversation Driver, which synthesizes multi-turn dialogues by alternating the user-proxy and assistant models; here, the assistant is also GPT-4o at `temperature=0`, ensuring low-variance responses for per-proxy comparability. Together, these blocks yield a self-contained manifest that our planner expands into (proxy, dataset, metric, seed) units, enabling exact replay and variance-aware aggregation across runs. We illustrate an example of such manifest file in Figure 17, which is a manifest produced using config shown in Figure 19.

Once the CLI command `mirrorbench run -c config.yaml` is invoked, the evaluation job starts and executes through all the units. Once the run is completed, another command `mirrorbench report json <run-name> –output report.json` can be executed to generate final evaluation report as shown in Figure 18.

```
{
  "run": {
    "run_id": "chatbot_arena_mirror-gpt-4o-user-gpt-4o-assistant-claude-4-sonnet-judge",
    "planner_version": "0.1.0",
    "engine": "sync",
    "created_at": "2025-10-25T04:55:52.1923690",
    "extra": {
      "status": "completed",
      "notes": []
    }
  },
  "aggregates": [
    {
      "metric_name": "metric:judge/gteval",
      "mean": 0.54875,
      "standard_deviation": 0.3000601907720393,
      "confidence_interval": 0.04158545659359925,
      "p_value": null,
      "sample_size": 200,
      "extra": {
        "rubric_version": "1.0",
        "judge_client": "client:langchain/chat",
        "prompt_version": "1.0",
        "total_proxy_user_turns": 243,
        "total_real_user_turns": 243,
        "bootstrap": null,
        "count": 200,
        "calibration": {
          "enabled": true,
          "raw_mean": 0.54875,
          "hh_mean": 0.99,
          "pp_mean": 0.99,
          "gap": 0.0
        }
      }
    },
    {
      "metric_name": "metric:judge/pi_pairwise",
      "mean": 0.4416666666666665,
      "standard_deviation": 0.4040381040237058,
      "confidence_interval": 0.05599579535494467,
      "p_value": null,
      "sample_size": 200,
      "extra": {
        "prompt_version": "1.0",
        "judge_client": "client:langchain/chat",
        "tie_count": 6,
        "sample_count": 600,
        "order_counts": {
          "HP": 315,
          "PH": 285
        },
        "control_counts": {
          "HH": 0,
          "PP": 0
        },
        "bootstrap": null,
        "count": 200,
        "parity_gap": -0.058333333333333315,
        "tie_rate": 0.01,
        "calibration": {
          "enabled": true,
          "raw_mean": 0.4416666666666665,
          "calibrated_mean": 0.445,
          "hh_mean": 0.4991666666666665,
          "pp_mean": 0.5008333333333334,
          "gap": -0.0016666666666667052
        }
      }
    }
  ],
  "units": [
    {
      "proxy_name": "proxy:langchain/gpt4o",
      "dataset_name": "dataset:jsonl/chatbot_arena_mirror",
      "metric_name": "metric:judge/gteval",
      "seed": 0,
      "judge_name": null
    },
    {
      "proxy_name": "proxy:langchain/gpt4o",
      "dataset_name": "dataset:jsonl/chatbot_arena_mirror",
      "metric_name": "metric:judge/pi_pairwise",
      "seed": 0,
      "judge_name": null
    }
  ],
  "telemetry": {
    ...
  },
  "notes": []
}
```


Figure 18: Final run report (JSON, abridged). Aggregate metric scores with 95% CIs, calibration controls (HH/PP), unit grid, and run metadata for the completed evaluation.

```
{
  "schema_version": "1.0",
  "created_at": "2025-10-25T04:39:21.5529522",
  "plan": {
    "user_proxies": [
      {
        "name": "proxy:langchain/gpt4o",
        "adapter": "adapter:generic/llm",
        "params": {
          "model_client": "client:langchain/chat",
          "client_params": {
            "model_import": "langchain_openai.ChatOpenAI",
            "model_kwargs": {
              "model": "gpt-4o",
              "temperature": 0
            }
          },
          "request_params": {}
        },
        "label": null
      }
    ],
    "datasets": [
      {
        "name": "dataset:jsonl/chatbot_arena_mirror",
        "split": "default",
        "params": {
          "path": "data/chatbot_arena/chatbot_arena_mirror.jsonl",
          "max_examples": 200
        },
        "task": null,
        "label": "Chatbot Arena Mirror"
      }
    ],
    "metrics": [
      {
        "name": "metric:judge/gteval",
        "params": {
          "judge_client_name": "client:langchain/chat",
          "judge_params": {
            "model_import": "langchain_anthropic.ChatAnthropic",
            "model_kwargs": {
              "model": "claude-sonnet-4-20250514",
              "temperature": 0
            }
          },
          "num_judge_samples": 1,
          "compute_controls": true
        },
        "label": "GTEval Realism",
        "judge": null
      },
      {
        "name": "metric:judge/pi_pairwise",
        "params": {
          "judge_client_name": "client:langchain/chat",
          "judge_params": {
            "model_import": "langchain_anthropic.ChatAnthropic",
            "model_kwargs": {
              "model": "claude-sonnet-4-20250514",
              "temperature": 0
            }
          },
          "num_judge_samples": 3,
          "compute_controls": true
        },
        "label": "Pairwise Indistinguishability",
        "judge": null
      }
    ],
    "metric_judges": {},
    "task_drivers": {
      "dataset:jsonl/chatbot_arena_mirror": {
        "name": "task:mirror/conversation",
        "params": {
          "assistant_model_client": "client:langchain/chat",
          "client_params": {
            "model_import": "langchain_openai.ChatOpenAI",
            "model_kwargs": {
              "model": "gpt-4o",
              "temperature": 0
            }
          },
          "request_params": {}
        }
      }
    },
    "config_hash": "382500147091f0a",
    "units": [
      {
        "proxy_name": "proxy:langchain/gpt4o",
        "dataset_name": "dataset:jsonl/chatbot_arena_mirror",
        "metric_name": "metric:judge/gteval",
        "seed": 0,
        "judge_name": null
      },
      {
        "proxy_name": "proxy:langchain/gpt4o",
        "dataset_name": "dataset:jsonl/chatbot_arena_mirror",
        "metric_name": "metric:judge/pi_pairwise",
        "seed": 0,
        "judge_name": null
      }
    ],
    "skipped": [],
    "planner_version": "0.1.0",
    "created_at": "2025-10-25T04:39:10.8455232"
  },
  "run_config": {
    "name": "chatbot_arena_mirror-gpt-4o-user-gpt-4o-assistant-claude-4-sonnet-judge",
    "seeds": {
      "0": 0
    },
    "engine": "async",
    "max_concurrency": 8,
    "timeout_seconds": 600.0,
    "max_retries": 0,
    "retry_backoff_seconds": 2.0,
    "cache": {
      "enabled": true,
      "ttl_seconds": 86400,
      "backend": "sqlite"
    },
    "judge": null,
    "observability": {
      "log_json": false,
      "log_level": "INFO",
      "log_destination": null,
      "tracing_enabled": false,
      "metrics_enabled": false,
      "otel_exporter": "otlp",
      "otel_endpoint": null
    },
    "metrics": {
      "bootstrap": null
    }
  }
}
```


Figure 17: Plan manifest (JSON, abridged). Fully resolved, replayable spec listing the (proxy, dataset, metric, seed) units, driver params, versions, and `config_hash`, enabling deterministic re-runs.

```yaml
run:
  name: chatbot_arena_mirror-gpt-4o-user-gpt-4o-assistant-sonnet-4-judge
  seeds:                                # Random seeds
  - 0
  engine: async                         # Selecting Backend
  max_concurrency: 8                    # Maximum concurrent workers
  timeout_seconds: 600
  cache:                                # Cache Config
    enabled: true
  observability:
    log_json: false
    log_level: INFO

user_proxies:                           # User-proxies to be evaluated
- name: proxy:langchain/gpt4o
  adapter: adapter:generic/llm
  params:
    model_client: client:langchain/chat
    client_params:
      model_import: langchain_openai.ChatOpenAI
      model_kwargs:
        model: gpt-4o
        temperature: 0
    request_params: {}

datasets:                               # Benchmarking datasets
- name: dataset:jsonl/chatbot_arena_mirror
  split: default
  label: Chatbot Arena Mirror
  params:
    path: data/chatbot_arena/chatbot_arena_mirror.jsonl
    max_examples: 200

metrics:                                # Evaluation metrics configuration
- name: metric:judge/gteval
  label: GTEval Realism
  params:
    judge_client_name: client:langchain/chat
    judge_params:
      model_import: langchain_anthropic.ChatAnthropic
      model_kwargs:
        model: claude-sonnet-4-20250514
        temperature: 0
    num_judge_samples: 1
    compute_controls: true
- name: metric:judge/pi_pairwise
  label: Pairwise Indistinguishability
  params:
    judge_client_name: client:langchain/chat
    judge_params:
      model_import: langchain_anthropic.ChatAnthropic
      model_kwargs:
        model: claude-sonnet-4-20250514
        temperature: 0
    num_judge_samples: 3
    compute_controls: true

task_drivers:                           # Task driver for synthetic rollout
  dataset:jsonl/chatbot_arena_mirror:
    driver: task:mirror/conversation
    params:
      assistant_model_client: client:langchain/chat
      client_params:
        model_import: langchain_openai.ChatOpenAI
        model_kwargs:
          model: gpt-4o
          temperature: 0
      request_params: {}
```

Figure 19: Job configuration (abridged). YAML that fully specifies a MirrorBench run: backend and concurrency, proxy/assistant clients, dataset binding, judge metrics (with calibration controls), and the mirror-conversation driver.